HARAMAYA UNIVERSITY

College of Computing and Informatics
Department of Computer Science

INDUSTRIAL PRACTICE REPORT

Design and Development of a Web-Based Dynamic Website and Content Management System for Oda Bultum University

(OBU-CMS)

Submitted as Part of the Industrial Internship Program
Bachelor of Science Degree in Computer Science (Individual Internship Project)

Prepared By	**NEDI USMAN HUSSEN**
Student ID	**2184/16**
Academic Advisor	**Mr. Kibru G.**
Internship Site	**Oda Bultum University - ICT Directorate, Chiro, Ethiopia**
Internship Period	**(45 days)**
Submission Date	**October 2026**

DECLARATION AND APPROVAL

I, **NEDI USMAN HUSSEN**, declare that this Industrial Practice Report is my own work. It describes the software development project that I carried out at the Information and Communication Technology (ICT) Executive Office of Oda Bultum University as part of the Industrial Practice Program of the Department of Computer Science, Haramaya University. All sources of information used in the report have been acknowledged in the References section.

This report has been submitted for examination with the approval of the following persons.

Role	Name	Signature	Date
Student	**NEDI USMAN HUSSEN (2184/16)**		
Academic Advisor	**Mr. Kibru G.**		
Host Supervisor (ICT Director)	**MR MURAD ABRAHIM**		
Department Head	**Mr. Alemu G.**		

ACKNOWLEDGEMENT

First and above all, praise be to Almighty Allah (Subhanahu Wa Ta'ala) for granting me the strength, patience, and capability to undertake and complete this industrial internship project. All success is by His grace alone - Alhamdulillah.

I would like to express my sincere gratitude to all those who supported and guided me throughout this industrial internship project.

First and foremost, I extend my heartfelt thanks to my Academic Advisor, **Mr. Kibru G.**, of the Department of Computer Science, College of Computing and Informatics, Haramaya University, for his unwavering academic guidance, constructive feedback, and continuous encouragement throughout the internship period. His dedication to student development made a lasting impact on my growth as a software developer.

I am deeply grateful to the **ICT Directorate of Oda Bultum University** for providing a supportive working environment, server infrastructure access, and technical supervision during the development of the OBU-CMS. Special appreciation is extended to **Mr. Murad Ibrahim**, Head of the Registrar's Office of Oda Bultum University, for providing invaluable domain knowledge, workflow insight, and patience during requirements gathering and User Acceptance Testing sessions. His openness to share institutional processes was essential to the quality of this project.

I also extend special thanks to our Department Head, **Mr. Alemu G.**, and the entire Department of Computer Science, Haramaya University, for designing a structured and meaningful industrial internship program that bridges theoretical computer science education with real-world software engineering practice.

Finally, I am deeply grateful to my family for their unconditional love, prayers, and moral support throughout this challenging and rewarding journey. Your encouragement carried me through every difficult moment.

NEDI USMAN HUSSEN

Department of Computer Science, Haramaya University

Internship Site: Oda Bultum University, Chiro, Ethiopia

ABSTRACT

This industrial practice report documents the design and development of a web-based dynamic website and Content Management System (OBU-CMS) for Oda Bultum University, undertaken during a 45-day internship at the university's ICT Directorate in Chiro, West Hararghe, Oromia, Ethiopia.

The existing web infrastructure of Oda Bultum University was a fragmented, template-based website (obu.edu.et) that depended on technical staff for content updates, had no structured content types, no media management, no role-based access control or audit trail, and limited mobile responsiveness. This created bottlenecks for the university's communications, admissions, and public information dissemination.

The proposed OBU-CMS replaces this system with a dynamic, database-driven web platform built on TanStack Start (React 19), TypeScript, Tailwind CSS v4, Supabase (PostgreSQL), and Radix UI. The system provides a public website with slug-based dynamic routing for the main content entities (news, events, announcements, academic programs, research projects, leadership profiles, services, and galleries) and an Admin CMS panel at /admin that lets authorized staff manage university content without writing code.

Key technical results include: Row Level Security (RLS) policies at the PostgreSQL layer; a centralized media library backed by Supabase Storage; server-side rendering (SSR) of public pages; CSRF middleware on all server functions; an activity audit log; and a normalized PostgreSQL schema of 27 tables managed through versioned migration files.

Testing combined 45 integration test cases run in Postman, Row Level Security checks of unauthenticated access, cross-browser and responsive checks, and seven User Acceptance Testing scenarios with the ICT Directorate and the Registrar's Office. A final code review also produced a list of remaining improvements (Chapter 5 and Chapter 6) that the ICT Directorate should address before wider production use.

Keywords: Content Management System, Web Development, TanStack Start, Supabase, PostgreSQL, Row Level Security, Server-Side Rendering, React, TypeScript, Tailwind CSS, Oda Bultum University, Industrial Internship.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CMS	Content Management System
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DB	Database
ERD	Entity-Relationship Diagram
FK / PK	Foreign Key / Primary Key
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
ICT	Information and Communication Technology
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LAN	Local Area Network
OBU	Oda Bultum University
OBU-CMS	Oda Bultum University Website and Content Management System
ORM	Object-Relational Mapping
QA	Quality Assurance
RBAC	Role-Based Access Control
REST	Representational State Transfer
RLS	Row Level Security (PostgreSQL)
SADD	System Architecture Design Document
SEO	Search Engine Optimization
SPA	Single Page Application
SQL	Structured Query Language
SSE	Server-Sent Events
SSR	Server-Side Rendering
UAT	User Acceptance Testing
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

TABLE OF CONTENTS

Acknowledgement..... i
Abstract..... ii
List of Abbreviations..... iii
Table of Contents..... iv
List of Figures and Tables..... v
Organization of the Document..... vii
CHAPTER 1: INTRODUCTION..... 2
1.1. Background of the Organization..... 2
1.2. Structure of the Organization..... 2
1.3. Team / Department Overview..... 4
1.4. Project Context..... 5
1.5. Objectives..... 6
CHAPTER 2: SOFTWARE DEVELOPMENT PLAN..... 9
2.1. Project Schedule..... 9
2.2. Resources..... 10
2.3. Risk Management..... 11
2.4. Development Methodology..... 12
2.5. Quality Assurance..... 13
2.6. Required Documents..... 13
CHAPTER 3: SYSTEM REQUIREMENTS..... 16
3.1. Existing System Analysis..... 16
3.2. Proposed System..... 19
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN..... 23
4.1. System Models..... 23
4.2. User Interface Design..... 34
4.3. Software Architecture..... 39
4.4. Security and Access Control..... 43
4.5. Database Design..... 45
CHAPTER 5: IMPLEMENTATION..... 50
5.1. Development Process..... 50
5.2. Tools Used..... 52
5.3. Challenges and Solutions..... 53
5.4. Testing..... 55
CHAPTER 6: INTERNSHIP REFLECTION AND CONCLUSION..... 59
6.1. Learning Outcomes..... 59
6.2. Challenges Faced..... 60
6.3. Recommendations..... 61
6.4. Conclusion..... 62
APPENDICES..... 63
Appendix A: Code Snippets..... 63
Appendix B: Database Tables Summary..... 64
Appendix C: Route Overview..... 65
Appendix D: Administrator Quick Guide..... 66
Appendix E: Test Script (`test_final_curls.sh`, excerpt)..... 66
REFERENCES..... 67

LIST OF FIGURES

Figure	Title
Figure 1.1	Organizational hierarchy of Oda Bultum University (relevant to OBU-CMS)
Figure 2.1	Project Gantt chart (45-day internship schedule)
Figure 3.1	Before - existing OBU website: lost-password page
Figure 3.2	Before - existing OBU website: President's message page
Figure 3.3	Before - existing OBU website: Administration drop-down over the home page
Figure 3.4	Before - existing OBU website: drop-down over the map and footer
Figure 4.1	Use case diagram of OBU-CMS
Figure 4.2	Class diagram of the OBU-CMS core domain
Figure 4.3	Class diagram of the main application modules
Figure 4.4	Object diagram - publishing a news article
Figure 4.5	Sequence diagram - administrator login and route protection
Figure 4.6	Sequence diagram - create and publish a news article
Figure 4.7	Sequence diagram - a visitor reads a published news article
Figure 4.8	State chart diagram - content item lifecycle
Figure 4.9	Activity diagram - admin content publishing workflow
Figure 4.10	Navigational flow - public website
Figure 4.11	Navigational flow - Admin CMS
Figure 4.12	After - new public website: home page and hero slider
Figure 4.13	After - new public website: President's message section
Figure 4.14	After - new public website: newsletter block and footer
Figure 4.15	After - Admin CMS: login page
Figure 4.16	After - Admin CMS: dashboard
Figure 4.17	After - Admin CMS: homepage management
Figure 4.18	After - Admin CMS: create user form
Figure 4.19	Three-tier software architecture
Figure 4.20	Component diagram
Figure 4.21	Deployment diagram
Figure 4.22	Entity-relationship diagram (1): security, settings, and web content
Figure 4.23	Entity-relationship diagram (2): academics, administration, research, and services

LIST OF TABLES

Table	Title
Table 1.1	People involved in the project
Table 1.2	Specific objectives and measurable outcomes
Table 2.1	Project schedule - phases, activities, and durations
Table 2.2	Risk management register
Table 3.1	Limitations of the existing website
Table 3.2	Functional requirements - public website
Table 3.3	Functional requirements - Admin CMS
Table 3.4	Non-functional requirements
Table 4.1	Primary use cases and actors
Table 4.2	Use case description: publish a news article
Table 4.3	Subsystems of OBU-CMS
Table 4.4	RBAC access matrix - role permissions
Table 4.5	Row Level Security policies in the migration files
Table 4.6	Security controls and where they are enforced
Table 4.7	Database tables by module
Table 5.1	Project folder structure
Table 5.2	Migration history
Table 5.3	Development tools by category
Table 5.4	Other challenges and solutions
Table 5.5	Integration testing results by module
Table 5.6	RLS security testing results
Table 5.7	Cross-browser and responsive testing results
Table 5.8	User Acceptance Testing scenarios and results
Table 6.1	Skills gained
Table 6.2	Real-world problems solved by the project
Table B.1	Database tables summary
Table D.1	Common administrator tasks

ORGANIZATION OF THE DOCUMENT

This industrial practice report is organized into the following chapters and preliminary sections, each addressing a distinct aspect of the internship and the OBU-CMS software development project:

- **Preliminary Sections:** the front page, cover page with declaration, acknowledgement, abstract (executive summary), list of abbreviations, table of contents, list of tables, list of figures, and this organizational overview.
- **Chapter 1 - Introduction:** the background of the host organization (Oda Bultum University), its organizational structure and hierarchy, the internship team and department overview, the project context (purpose, problem statement, and scope), and the general and specific objectives of the OBU-CMS project.
- **Chapter 2 - Software Development Plan:** the project schedule with a Gantt chart, required hardware and software resources, the physical workspace, the risk management register, the development methodology (Agile Scrum), quality assurance protocols, and the list of required project documents.
- **Chapter 3 - System Requirements:** the analysis of the existing website and the full functional and non-functional requirements of the proposed OBU-CMS.
- **Chapter 4 - System Analysis and Design:** all system models (use case, class, object, sequence, state chart, and activity diagrams), the user interface navigational flow, the software architecture (three-tier architecture, component diagram, and deployment diagram), the security and access control design, and the complete database ERD.
- **Chapter 5 - Implementation:** the sprint-by-sprint development process, the tools and frameworks used, the major technical challenges and their solutions, and the testing results (integration, security, cross-browser, and UAT).
- **Chapter 6 - Internship Reflection and Conclusion:** the learning outcomes (practical and theoretical), the workplace and technical challenges faced, and recommendations for the ICT Directorate and for future Haramaya University internship programs.
- **Appendices and References:** key code snippets, a database tables summary, administrator quick guide, and all cited sources in IEEE style.

1

CHAPTER 1

Introduction

Background, organizational context, project purpose and objectives

Sections in this Chapter

- 1.1 Background of the Organization
- 1.2 Structure of the Organization
- 1.3 Team and Department Overview
- 1.4 Project Context
- 1.5 Objectives

CHAPTER 1: INTRODUCTION

This chapter provides a comprehensive overview of the industrial internship undertaken at Oda Bultum University (OBU) in Chiro, West Hararghe, Oromia, Ethiopia. It describes the host organization, its structural hierarchy, the department in which the internship was conducted, the context of the software project, and the objectives set for the internship period.

1.1. Background of the Organization

Oda Bultum University (OBU) is a public higher education institution established in 2015 under the Ethiopian Ministry of Education. It is located in Chiro, the capital of the West Hararghe Zone of the Oromia Regional State, about 326 kilometres east of Addis Ababa. The university describes itself as one of the fourth-generation universities of Ethiopia and was founded to expand access to quality higher education in the eastern regions of the country, addressing a gap in the availability of tertiary institutions serving a large and historically underserved population. Its motto, stated by the President in the welcome message, is "Productive Mind for Development".

Since its establishment, OBU has grown in enrollment, academic programs, and physical infrastructure. The university serves thousands of undergraduate students across six academic units: the College of Agriculture, the College of Natural and Computational Sciences, the College of Business and Economics, the College of Social Sciences and Humanities, the College of Health and Medical Sciences, and the Institute of Technology. These offer undergraduate programs in regular, extension, weekend, and summer modalities, serving both school leavers and working professionals.

The university operates under the leadership of a President, supported by Vice Presidents. As the institution has grown, the need for a modern, dynamic, and centrally managed web presence has become urgent. OBU's existing website (obu.edu.et) was fragmented and required technical intervention for content updates, creating bottlenecks for communications, admissions, and public information dissemination.

The **ICT Directorate** (the ICT Executive Office in the university's structure) is the host organization of this internship. During the internship, the intern worked in close collaboration with the ICT Directorate and the university's communications teams to investigate the shortcomings of the existing website and to design and develop a modern replacement: the **OBU-CMS**.

1.2. Structure of the Organization

Oda Bultum University operates under a hierarchical administrative structure consistent with the Ethiopian public university governance framework. As shown in the Administration menu of the existing university website, the **University President** is the chief executive officer, responsible for institutional leadership, strategic direction, and external representation. Three main branches report to the President:

- **President Offices:** the executive offices and directorates directly attached to the President: the Information and Communication Technology (ICT) Executive Office, the Internal Audit Office, the Strategic Planning, Monitoring and Evaluation Executive Office, the Women, Children and Youth Affairs Directorate, the Legal Service Directorate, the Public and International Relations Directorate, the Construction Project Office, and the University Protection and Security Service Directorate.
- **Administrative and Development Vice President:** responsible for administrative, financial, and development support services of the university.
- **Academic, Research, Technology Transfer and Community Service Vice President:** oversees academic programs, faculty matters, curriculum policy, research, technology transfer, community service, and the Registrar's Office.

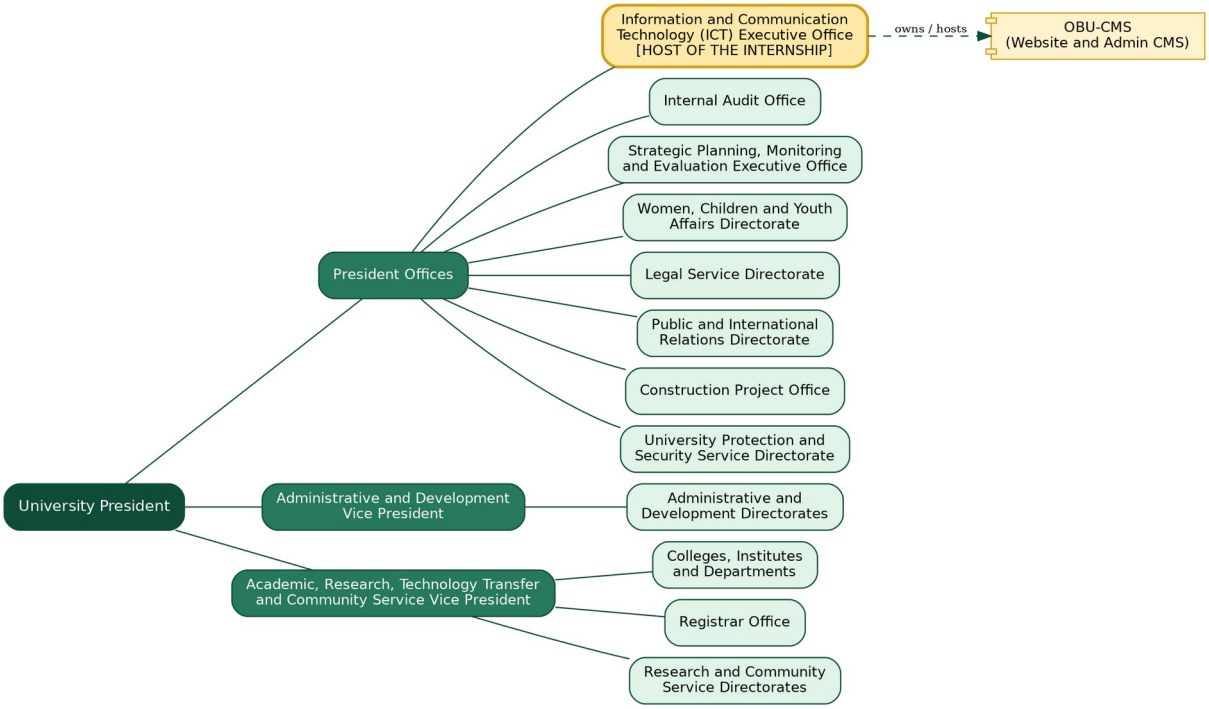


Figure 1.1: Organizational hierarchy of Oda Bultum University (relevant to OBU-CMS)

The following units and groups are directly relevant to the OBU-CMS project:

- **ICT Directorate:** manages the university's information technology infrastructure, including servers, network, and software systems. It acts as the technical owner and deployment host of the OBU-CMS.

- **Registrar's Office:** the central body for student admissions, enrollment, and academic records. The Registrar section of OBU-CMS exposes downloadable documents and announcements to students.
- **Research and Community Service units:** coordinate research centers, publications, and community engagement programs, all of which are content modules in OBU-CMS.
- **College Deans and Department Heads:** academic heads whose profiles, programs, and departmental information are managed through the Admin CMS.
- **Public and International Relations / Communications staff:** supply news, events, and announcements.
- **Students and the General Public:** the primary end-users of the public website, who read news, events, programs, announcements, and services.

This hierarchy directly informs the access control design of the system. The ICT Directorate manages administrator accounts. Content editors from each office manage their own modules through the Admin CMS. In practice, the ICT Director acts as System Administrator (Super Admin), communications staff act as Content Editors, Registrar officers manage Registrar-specific modules, and the general public has read-only access to published content.

1.3. Team / Department Overview

The internship was conducted primarily within the **ICT Directorate** of Oda Bultum University. The Directorate is the technical backbone of the institution, managing the university's LAN, server infrastructure, and all deployed software systems. It is staffed by IT officers, network engineers, and system administrators who maintain uptime, manage user accounts, and provide technical support across all colleges and offices.

The intern worked under the supervision of the ICT Director and collaborated closely with the following people:

Table 1.1: People involved in the project

Person / Office	Contribution to the project
ICT Director (host supervisor)	Provided institutional guidance, approved the project scope, coordinated access to server infrastructure, and reviewed technical deliverables before each sprint review.
Senior Web and Content Officer	Primary domain expert: explained the existing website structure, content workflows, and the pain points of content managers; took part in User Acceptance Testing.
Communications and PR Office representative	Took part in requirements sessions and specified how news, events, and announcements should be structured and displayed.
Mr. Murad Ibrahim, Head of the Registrar's Office	Provided Registrar domain knowledge and workflow insight; took part in requirements gathering and User Acceptance Testing.
Academic Advisor, Mr. Kibru G. (Haramaya University)	Academic oversight, review of deliverables, and alignment with the requirements of the Department of Computer Science.

Person / Office	Contribution to the project
Intern (Nedi Usman Hussen)	Sole full-stack developer: requirements analysis, architecture and database design, backend and frontend development, Admin CMS, testing, deployment support, and documentation.

1.3.1. Daily Workflow and Collaboration Methods

Collaboration followed a structured cadence throughout the 45-day internship. Progress meetings were held with the ICT Director every Monday and Thursday to review completed sprint tasks, demonstrate working increments, and obtain approval for the next sprint's scope. Ad-hoc requirement validation sessions were held with the Senior Web and Content Officer whenever content structure decisions arose. Written progress summaries were submitted to the Academic Advisor, Mr. Kibru G., at the end of each sprint.

The ICT Directorate's internal workflow follows a ticket-based system for user support requests, while infrastructure changes require written approval from the ICT Director before deployment. During the internship, all system changes were logged and reviewed with the ICT Director before each sprint review, which kept the work aligned with the Directorate's change management process.

1.4. Project Context

1.4.1. Purpose

The central purpose of this internship project is to replace the fragmented and manually maintained web presence of Oda Bultum University with a unified, modern, and high-performance web-based dynamic website and Content Management System (OBU-CMS). The system enables the university to publish and manage all public-facing content, including news, events, announcements, academic programs, research projects, leadership profiles, services, and gallery media, through a secure, role-based Admin CMS panel, without direct code modifications.

1.4.2. Problem Statement

OBU's existing web infrastructure suffered from several limitations identified through observation and stakeholder interviews during the initial phase of the internship:

- **Dependence on technical staff for content.** Publishing or correcting information required help from the ICT staff, which delayed time-sensitive communications such as registration and exam notices.
- **No centralized, role-based content management.** Staff from different departments could not independently publish or update their own content, and content control was not separated by role.

- **No structured content types or dynamic routing.** News, events, announcements, vacancies, and programs were not separate managed records with their own addresses (slugs), so linking to a specific article, event, or research project was difficult.
- **No media management system.** Images and documents were not stored in a library with metadata, version tracking, or access control.
- **Poor mobile responsiveness.** The site was not designed mobile-first, so it was hard to use on smartphones and tablets.
- **No activity logging.** There was no audit trail of who published, edited, or deleted content, which created accountability gaps.
- **Outdated and inconsistent presentation.** Some pages had low-contrast, overflowing text blocks, the navigation was a deep mega-menu, and fixed text such as the footer copyright year (2023) was not updated automatically (see Chapter 3).

1.4.3. Scope

The scope of OBU-CMS covers the complete public-facing digital presence of Oda Bultum University, providing a dynamic, database-driven website together with an Admin Content Management System. The data, authentication, and file storage are hosted on a cloud Supabase PostgreSQL project, so that students, staff, and the public can reach up-to-date information from any device.

In scope - public website: home page (hero slides, statistics, welcome and President's message, featured content); news; events; announcements; academic units, colleges, departments, and programs; leadership; administrative offices; research projects, facilities, and community engagement; services; career vacancies; galleries; Registrar information, announcements, and documents; contact form; general pages and the privacy policy.

In scope - Admin CMS: secure login; dashboard with statistics and recent activity; create, read, update, and delete (CRUD) for the content modules; media library; page, navigation, and homepage-section management; user and role management; activity logs; site and system settings; personal account page.

Out of scope: a student portal (grades, course registration), online payments, mobile applications, and e-learning integration. These are listed as future work.

1.5. Objectives

1.5.1. General Objective

To design, develop, test, and deploy a secure, high-performance, web-based dynamic website and Content Management System (OBU-CMS) for Oda Bultum University that enables non-technical staff to manage university content independently, gives the public a modern and

mobile-responsive information portal, and removes the bottlenecks caused by the existing website.

1.5.2. Specific Objectives

The following specific objectives define the measurable outcomes of the OBU-CMS project by the end of the 45-day internship period:

Table 1.2: Specific objectives and measurable outcomes

No.	Specific objective	Measurable outcome
1	Replace the existing website with a dynamic, database-driven web application with slug-based routing for the content entities (news, events, programs, research, leadership, services, galleries).	Public route files for all planned modules; detail pages such as /news/$slug.
2	Build a secure, role-protected Admin CMS panel at /admin so that authorized staff can create, read, update, and delete content without writing code.	35 admin route files covering the main content modules, protected by login.
3	Implement role-based access control using Supabase Row Level Security (RLS) so that public users can read only published content while authenticated administrators can write.	RLS policies and storage policies in migration files; System menu limited to Super Admin.
4	Implement a centralized Media Library backed by Supabase Storage so that images and documents are uploaded once and reused across all content entities.	media table; upload validation (type, 10 MB); foreign keys from content tables.
5	Provide a fully responsive, mobile-first UI using Tailwind CSS v4.	Layouts verified on desktop, tablet, and mobile widths (Table 5.7).
6	Achieve fast initial page loads (target: about 2 seconds) using TanStack Start server-side rendering, improving SEO and user experience.	SSR route loaders; page titles and descriptions per route.
7	Implement a comprehensive activity audit log recording admin actions (create, update, delete) with user identity and timestamp.	activity_logs table written by the data hooks.
8	Produce technical documentation: a system architecture design document, API documentation, and an administrator user manual for the ICT Directorate.	README, specification documents, migrations, this report, and Appendix D.

2

CHAPTER 2

Software Development Plan

Project schedule, resources, risk management, methodology, quality assurance

Sections in this Chapter

- 2.1 Project Schedule
- 2.2 Resources
- 2.3 Risk Management
- 2.4 Development Methodology
- 2.5 Quality Assurance
- 2.6 Required Documents

CHAPTER 2: SOFTWARE DEVELOPMENT PLAN

This chapter presents the software development plan of the OBU-CMS project: the schedule, the required resources, the risk management strategy, the development methodology, the quality assurance approach, and the documentation that was produced.

2.1. Project Schedule

The OBU-CMS internship project ran over a 45-day (approximately 9-week) execution window, from **[START DATE]** to **[END DATE]**. The Gantt chart and the schedule table below map the eight phases to their weeks and deliverables. After the internship period the system was refined, and the final database migrations (25 August to 22 September 2026), the final screenshots (8 October 2026), and this report were completed in the following months.

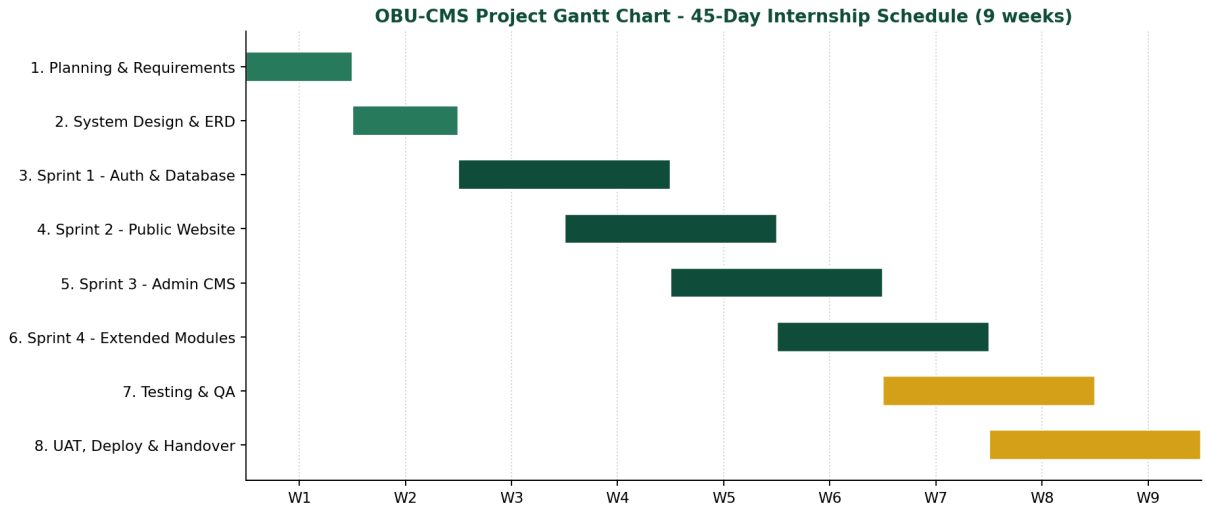


Figure 2.1: Project Gantt chart (45-day internship schedule)

Table 2.1: Project schedule - phases, activities, and durations

Phase	Activity / deliverable	Weeks	Duration
1. Planning	Stakeholder interviews, requirement gathering, user story writing, product backlog.	Week 1	5 days
2. System Design	Database schema (ERD), system architecture, UI wireframes and mockups.	Week 2	5 days
3. Sprint 1 - Auth & DB	Supabase project setup, PostgreSQL schema migrations, RLS policies, admin authentication, dashboard skeleton.	Weeks 3-4	10 days
4. Sprint 2 - Public Web	Homepage, News, Events, Announcements, Programs, Leadership, About pages with SSR and dynamic slug routing.	Weeks 4-5	7 days
5. Sprint 3 - Admin CMS	CRUD admin panels for the content modules, Media Library, Navigation management.	Weeks 5-6	7 days
6. Sprint 4 - Extended	Research Projects, Facilities, Community Engagement, Services, Vacancies, Registrar section, Activity Logs.	Weeks 6-7	7 days

Phase	Activity / deliverable	Weeks	Duration
7. Testing & QA	Integration testing, RLS security audit, performance checks, bug fixes, cross-browser validation.	Weeks 7-8	5 days
8. UAT, Deploy & Handover	UAT with the ICT Director and content staff, deployment, documentation handover, training.	Weeks 8-9	4 days

Note: the schedule was adjusted slightly according to stakeholder availability and sprint review feedback.

2.2. Resources

2.2.1. Software Tools

- **TanStack Start (React 19):** the full-stack framework for the public website and Admin CMS. It provides SSR, file-based routing, and server functions (`createServerFn`) for the backend layer.
- **TanStack Router:** file-based routing with type-safe route parameters for all dynamic slug routes (for example `/news/$slug`, `/academics/programs/$slug`).
- **TanStack Query:** server-state management for data fetching, caching, and refreshing in the Admin CMS.
- **TypeScript:** the programming language, giving static type safety across the codebase.
- **Tailwind CSS v4:** a utility-first CSS framework for the responsive, mobile-first interface.
- **Supabase:** backend-as-a-service providing PostgreSQL hosting, JWT-based authentication, Row Level Security, and Supabase Storage for the media library.
- **node-postgres (pg):** the PostgreSQL client used by server functions for parameterized SQL.
- **Radix UI:** accessible UI primitives (modals, dropdowns, dialogs) for the Admin CMS components.
- **React Hook Form and Zod:** form state management, and a schema validation library that is installed for form and server-function validation.
- **Vite:** the build tool and development server.
- **Git / GitHub, VS Code, Postman:** version control, the IDE, and API testing during development.

The exact versions are listed in Table 5.3 (Chapter 5).

2.2.2. Hardware and Infrastructure

- **Development machine:** a personal laptop running Kali Linux, used for development, testing, and code management.

- **Supabase Cloud:** the cloud-hosted PostgreSQL instance providing the database, authentication service, and file storage, reachable over the internet with RLS policies enforcing access boundaries.
- **Campus LAN / Internet:** access to the system over the internet, so that students and the public can reach the website from any device. Admin CMS access is protected by Supabase authentication.

2.2.3. Physical Workspace

The intern was given a designated workstation in the OBU ICT Directorate main office, located in the university's administrative building. The workspace had reliable power, direct campus LAN connectivity with internet access, and access to the university's internal server infrastructure, which allowed continuous work on development and staging environments throughout the internship.

2.2.4. Developer Role (Individual Project)

This project is an individual effort. Requirements analysis, database design, system architecture, backend server function development, frontend implementation, Admin CMS development, testing, documentation, and deployment support were carried out by the intern, **Nedi Usman Hussen**, under the guidance of supervisors. The other roles are listed in Table 1.1: the ICT Director provides server access, guidance, and deployment oversight but does not contribute to development; the Senior Web and Content Officer provides domain knowledge and takes part in UAT; and the Academic Advisor, Mr. Kibru G., provides academic oversight and feedback.

2.3. Risk Management

A risk register was established at the start of the project. The table below shows each key risk, its probability, its impact, and the planned mitigation and contingency.

Table 2.2: Risk management register

Risk	Prob.	Impact	Mitigation / contingency
Scope creep: stakeholders request additional content modules	High	High	Formal change requests; new modules deferred to a post-internship backlog.
Supabase service outage during development and testing	Medium	High	Local development using a direct PostgreSQL connection through the DATABASE_URL setting in .env.
Data migration loss when moving existing content into the new database	Medium	Critical	Content entered through the Admin CMS and verified page by page; row counts compared before cutover.
RLS policy misconfiguration exposing admin-only data to the public	Low	Critical	Policies reviewed with the ICT Director; tests send unauthenticated requests; check_rls.ts and query_policies.js inspect the real policies.
Key stakeholders unavailable during UAT	Medium	Medium	UAT sessions scheduled weeks in advance with backup reviewers.

Risk	Prob.	Impact	Mitigation / contingency
Performance degradation under concurrent public traffic	Medium	High	SSR reduces client-side load; TanStack Query caching reduces repeated queries.
Timeline slippage because Admin CMS modules are more complex than estimated	Medium	Medium	Schedule buffer; non-critical modules deferred if needed; reusable hook and form patterns.
Security weakness in the authentication layer allowing unauthorized admin access	Low	Critical	Supabase Auth (JWT) with RLS at the database layer; CSRF middleware on server functions.
Database structure changing while pages depend on it (schema drift)	High	Medium	All changes written as migration files; typed definitions in `database.types.ts` updated after each change.
Test or development credentials left in the project	Medium	High	Development-only admin script clearly marked; recommendation to remove it and rotate the password before production (Chapter 6).
Accidental data deletion by an administrator	Low	High	Delete actions confirmed in the interface and written to the activity log; regular export recommended.

2.4. Development Methodology

The OBU-CMS project was developed using the **Agile Scrum** methodology, adapted to a single developer. This choice is justified by the following factors:

- **Evolving requirements:** different offices (Registrar's Office, Research and Community Service, Communications) had distinct and partly undefined needs that became clear gradually. Iterations let requirements be refined across sprints without disrupting the overall plan.
- **Frequent stakeholder feedback:** the ICT Director and the Content Officer were available for regular sprint reviews, so misaligned features could be corrected early.
- **Modular delivery:** the system divides naturally into independently deliverable modules (Public Website, Admin Auth, News/Events CMS, Research CMS, Media Library), each fitting a sprint.
- **Risk reduction:** a working increment at the end of each sprint means that a functional system exists at all times.

A strict Waterfall model was rejected because it would have fixed the requirements before the users had seen anything. The Scrum framework was applied with the following structure:

- **Sprint duration:** one to two weeks per sprint with a defined sprint goal.
- **Sprint planning:** at the start of each sprint, user stories were selected from the product backlog and broken into technical tasks.
- **Daily progress tracking:** short written progress notes tracked task completion against the sprint goal.
- **Sprint review:** a live demonstration to the ICT Director and the Content Officer at the end of each sprint for feedback and acceptance.

- **Retrospective:** a short reflection after each sprint to improve the next one.

A lightweight specification was also written before coding (requirements, design, and task list in specs/university-admin-media), and database changes were written as ordered migration files, so the iterative work stayed controlled.

2.5. Quality Assurance

Quality assurance was built into the whole development lifecycle and not left to the final stage.

2.5.1. Code Quality

- **Self code review with supervisor spot-checks:** every feature was reviewed against a checklist covering naming, avoiding repetition, error handling, and security patterns. Security-critical parts, in particular the RLS policies and the authentication logic, were reviewed with the ICT Directorate's system administrators.
- **Linting and formatting:** the TypeScript and React code is checked with ESLint and formatted with Prettier (npm run lint, npm run format).
- **Version control discipline:** work was tracked in Git with descriptive commits and feature branches.

2.5.2. Testing

- **Static checking:** the TypeScript compiler with typed table definitions finds wrong column names and data shapes at build time.
- **Integration testing:** server function endpoints were tested end-to-end with Postman to verify the data flow between the frontend, the server functions, and PostgreSQL; curl scripts check that data created in the Admin CMS appears on the public pages.
- **RLS security testing:** each protected table was tested with unauthenticated and incorrectly authenticated requests to verify the access boundaries.
- **Cross-browser and responsive testing:** public pages and the Admin CMS were checked on Chrome, Firefox, and Safari at desktop (1280 px), tablet (768 px), and mobile (375 px) widths.
- **User Acceptance Testing (UAT):** carried out with representative users from the stakeholder groups in Week 8; feedback was recorded, prioritized, and handled before deployment.
- **Automated unit tests:** a unit-test framework was not set up during the internship; this is listed as an improvement in Chapter 6.

2.5.3. Documentation Standards

- Server functions and hooks are documented with comments describing their inputs, outputs, and security constraints.

- A System Architecture Design Document (SADD) covers the database schema (ERD), API design, RLS role matrix, and deployment topology; its content is Chapter 4 of this report.
- Document versions follow a simple version number (v1.0, v1.1, v2.0), and the final documentation package is tagged in Git.

2.6. Required Documents

The following documentation was planned and prepared for delivery to the ICT Directorate of Oda Bultum University:

1. **System Architecture Design Document (SADD):** the three-tier architecture, ERD, API design, RLS role matrix, Supabase Storage structure, and deployment topology diagram (Chapter 4).
2. **Database Schema Document:** the entity-relationship diagrams with table definitions, foreign key constraints, RLS policies, and migration history (Chapter 4 and 5, supabase/migrations/).
3. **API Documentation:** comments for the server functions covering inputs, return types, authentication requirements, and errors.
4. **Administrator's Manual:** a step-by-step guide for the ICT Directorate on system access, Admin CMS navigation, content modules, the media library, user and role management, and the activity log (Appendix D).
5. **Content Editor User Manual:** a role-specific guide for non-technical staff on publishing news, creating events, managing announcements, and uploading media (Appendix D).
6. **Source Code Repository:** commented source code in a Git repository with a README, environment settings, migration instructions, and setup guide.
7. **Test Reports:** the Postman integration results, the RLS security audit, and the cross-browser and responsive checks (Chapter 5).
8. **Industrial Practice Report:** this document, submitted to the Department of Computer Science of Haramaya University as partial fulfillment of the B.Sc. program requirements.

3

CHAPTER 3

System Requirements

Existing system analysis and functional / non-functional requirements

Sections in this Chapter

- 3.1 Existing System Analysis
- 3.1.1 Visual Comparison: Before and After
- 3.2 Proposed System
- 3.2.1 Functional Requirements
- 3.2.2 Non-Functional Requirements

CHAPTER 3: SYSTEM REQUIREMENTS

This chapter analyzes the existing website of Oda Bultum University and presents the complete functional and non-functional requirements of the proposed OBU-CMS system.

3.1. Existing System Analysis

Before this internship project, the web presence of Oda Bultum University was the website at obu.edu.et, a template-based, WordPress-style site managed by the ICT Directorate. It gave general information about the university (college listings, leadership, and contact details), with a green header and contact bar, a main menu (Home, About Us, Academics, Administration, Registrar, Research, Services, Vacancy), an image slider, a welcome section, a Google map, and a footer. The Administration menu is a three-level drop-down that lists the President offices and the two Vice Presidencies. The site also exposes a login and lost-password page. The following limitations were identified through observation and interviews with the ICT and content staff:

- **Dependence on technical staff:** publishing or correcting content needed help from technical staff; a single update could take hours, which delayed time-sensitive notices (registration, exams).
- **No structured content types:** news, events, announcements, vacancies, and programs were not separate managed records with status, date, image, and a clean address, so linking to a specific item was difficult and content could not be searched or filtered reliably.
- **No media management:** photos and documents were not stored in a library with metadata, alt text, or search.
- **No role separation:** content control was not divided between administrators, content editors, and Registrar staff.
- **Exposed account pages:** the lost-password page was publicly visible (Figure 3.1) without any visible rate-limiting or other protection.
- **Deep and fragile navigation:** the Administration mega-menu has three levels and overlays the page content, which is hard to use on small screens (Figure 3.3).
- **Poor readability and outdated text:** some pages show low-contrast, overflowing text blocks (Figure 3.2), and the footer shows a fixed copyright year (2023) and fixed contact lines (Figure 3.4).
- **Weak mobile optimization and SEO:** pages were not designed mobile-first and did not use consistent titles and descriptions.
- **No activity logging:** there was no record of who changed what and when.

Table 3.1: Limitations of the existing website

No.	Limitation	Evidence / effect
1	Content changes depend on technical staff	Reported by ICT and content staff; time-sensitive notices are delayed.
2	Deep navigation	Three levels and eight sub-items under President Offices alone (Figure 3.3).
3	Outdated, fixed text	Fixed copyright year 2023 and fixed contact lines (Figure 3.4).
4	Poor readability on some pages	President's message page with weak contrast and overflowing text (Figure 3.2).
5	No structured content types	No managed records with status, date, image, and clean address.
6	No central media library	Files cannot be reused or searched with descriptions.
7	No role separation and no audit trail	Staff cannot be limited to their own modules; changes are not recorded.
8	Exposed lost-password page	Publicly visible without visible protection (Figure 3.1).

3.1.1. Visual Comparison: Before and After

The following screenshots show the contrast between the old OBU website (obu.edu.et) and the newly developed OBU-CMS. The old site was a fragmented WordPress-style system with a deep drop-down navigation, an exposed password reset form, limited mobile responsiveness, and no structured content management for non-technical staff.

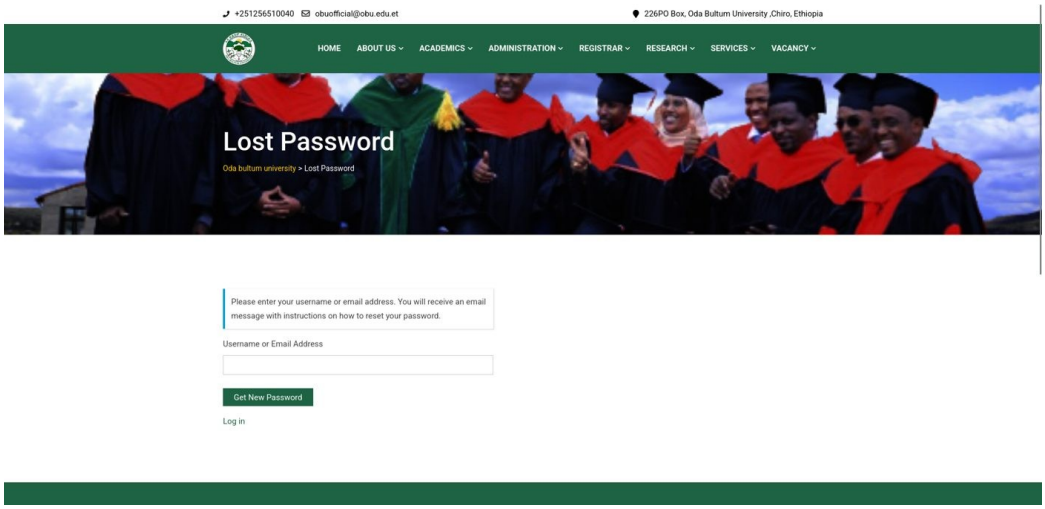


Figure 3.1: Before - existing OBU website: (a) lost-password page exposed to the public

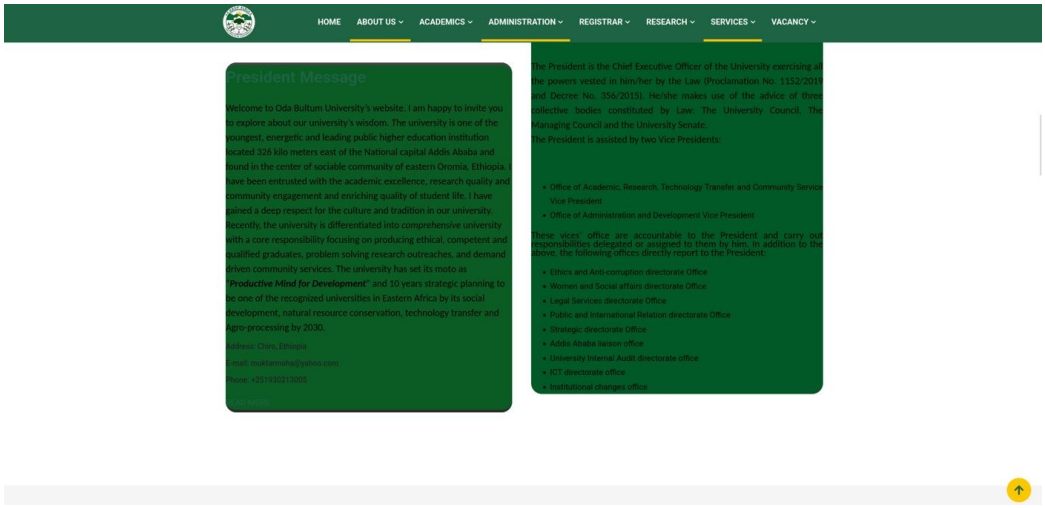


Figure 3.2: Before - existing OBU website: (b) President's message with fragmented two-column layout and low-contrast text

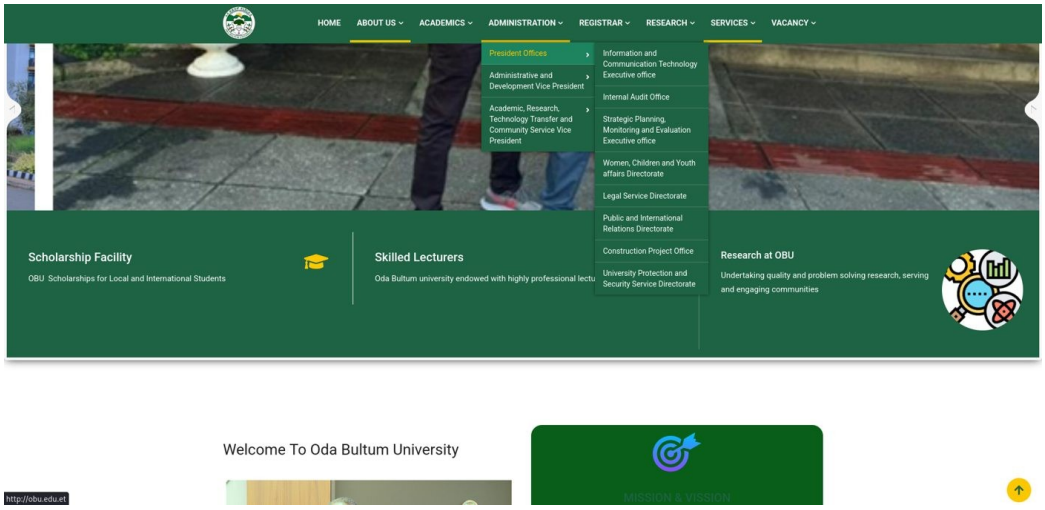


Figure 3.3: Before - existing OBU website: (c) Administration drop-down over the home page

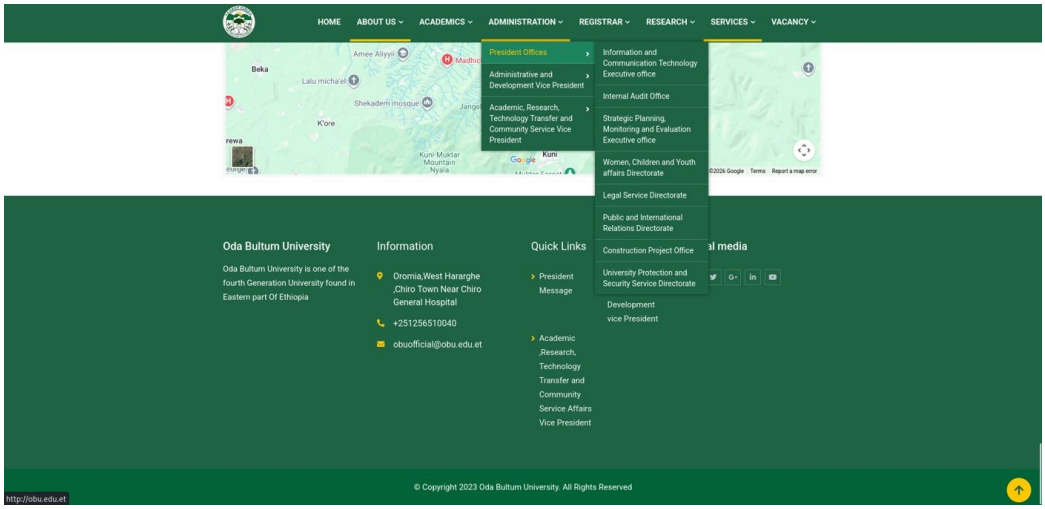


Figure 3.4: Before - existing OBU website: (d) Administration drop-down over the map and footer (copyright text fixed to 2023)

The corresponding screens of the new OBU-CMS public website and Admin panel are shown in Chapter 4 (Figures 4.12 to 4.18): the dynamic home page with hero slider, the President's

message section with photo and biography, the footer navigation, the admin login, the dashboard with live statistics, homepage management, and user creation with role assignment.

3.2. Proposed System

The proposed system, OBU-CMS, is a single web application with two parts: a **public website** for visitors and an **Admin CMS** for authorized staff. Both read and write the same PostgreSQL database hosted on Supabase.

3.2.1. Functional Requirements

Public website requirements:

Table 3.2: Functional requirements - public website (FR-01 to FR-16)

ID	Functional requirement
FR-01	Display a dynamic homepage with hero slides, statistics, featured news, upcoming events, and research highlights fetched from the database.
FR-02	Display a news listing page with search and category filtering and individual article detail pages accessible via unique URL slugs.
FR-03	Display upcoming events with date, time, location, and a detail page per event.
FR-04	Display published announcements, including Registrar-specific announcements, with expiry date support.
FR-05	Display academic programs organized by college and degree level with individual program detail pages.
FR-06	Display academic department/unit profiles with their associated programs.
FR-07	Display university leadership profiles with biography and contact information.
FR-08	Display research projects, research facilities, and community engagement programs.
FR-09	Display university services with location, hours, and contact information.
FR-10	Display open career vacancies with application deadline, requirements, and responsibilities.
FR-11	Display photo galleries with individual gallery detail pages.
FR-12	Provide a Registrar section with downloadable documents and Registrar-specific announcements.
FR-13	Provide a contact form that saves messages to the database with status tracking.
FR-14	Support general pages (About, Privacy Policy) managed through the CMS.
FR-15	Display a President's message section with photo and biography from the site settings and homepage content.
FR-16	Build the header menu and footer links from the navigation table.

Admin CMS requirements:

Table 3.3: Functional requirements - Admin CMS (FR-17 to FR-30)

ID	Functional requirement
FR-17	Provide a secure login page for admin users using email and password authentication through Supabase Auth.
FR-18	Display an admin dashboard showing content statistics, quick actions, pending actions, and recent activity.
FR-19	Provide create, view, edit, and delete interfaces for the public content modules (the admin screens for research facilities, community engagement, and the research gallery are linked in the sidebar but are still to be built).
FR-20	Provide a media library where admins can upload images and documents to Supabase Storage and attach them to any content entity.

ID	Functional requirement
FR-21	Allow admins to set a content status (Draft or Published; Active or Inactive) that controls public visibility. Review and Archived states are planned (see Section 4.1.5).
FR-22	Automatically generate a URL slug from the title for new content and add a number when the slug already exists.
FR-23	Provide user management so that a Super Admin can create users, assign roles, and activate or deactivate accounts.
FR-24	Provide role management: view roles and their permissions.
FR-25	Provide navigation management for the public website's menu structure.
FR-26	Provide homepage section management (hero slides, welcome, statistics, featured content) with show/hide and ordering.
FR-27	Log create, update, delete, and upload actions to an activity log table with user identity, timestamp, and description.
FR-28	Provide website settings management (university name, logo, contact information, social links, mission and vision text).
FR-29	Let each administrator edit their own profile (name, phone, profile image) and read contact messages sent from the public form.
FR-30	Automatically redirect unauthenticated users from all /admin/* routes to the login page.

3.2.2. Non-Functional Requirements

Table 3.4: Non-functional requirements

Category	Requirement	How it is met
Usability	The Admin CMS shall follow consistent design patterns across modules (same table layout, form structure, and action buttons) to reduce the learning curve. Form fields shall have clear labels and placeholders. Toast notifications shall give immediate feedback after every save or delete.	Same list-and-form pattern for every module; Sonner toast messages; sidebar groups by topic.
Performance	Public pages should fully render (SSR) within about 2 seconds on a standard broadband connection; Admin list views should load within about 3 seconds for tables of up to 500 records; queries should use indexed columns (slug, status).	Server-side rendering of route data; unique slug indexes; lists loaded with one query plus joins. These are targets and were observed informally, not measured with a tool.
Reliability	Data must stay consistent and failures must not show raw errors to visitors. Write operations that change several tables should be atomic.	Managed Supabase cloud infrastructure; foreign keys and unique constraints; error middleware and a friendly error page.
Security	Admin routes protected by Supabase Auth session verification; RLS enforced at the database layer; CSRF middleware on server functions; credentials stored in environment variables and not committed to version control.	See Section 4.4. Server-function input validation with Zod is listed as an improvement (Chapter 6).
Scalability	A stateless SSR server plus a cloud-hosted database should allow the server layer to scale horizontally without schema changes; media files should not depend on local server storage.	Supabase Storage for media; no local file storage.
Responsiveness	Pages must work on phones, tablets, and desktops.	Tailwind CSS responsive classes; collapsible admin sidebar.
Maintainability	The system should be easy to extend with new modules.	TypeScript types for every table; one data hook per module; migrations for each schema change.

Category	Requirement	How it is met
Legal compliance	A Privacy Policy page shall be included and the contact form shall collect only the minimum personal data (name, email, subject, message).	/privacy-policy page; contact form with those fields (phone optional).

4

CHAPTER 4

System Analysis and Design

UML diagrams, architecture, security model, and database design

Sections in this Chapter

- 4.1 System Models (Use Case, Class, Object, Sequence, State Chart, Activity)
- 4.2 User Interface Design and Navigational Flow
- 4.3 Software Architecture (Three-Tier, Component, Deployment)
- 4.4 Security and Access Control
- 4.5 Database Design (ERD)

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

This chapter presents the system models, the user interface design, the software architecture, the security model, and the database design of OBU-CMS. All diagrams were drawn from the real project: its route files, data hooks, migration files, and database types.

4.1. System Models

4.1.1. Use Case Diagram

OBU-CMS has three primary actor categories. The **Public User (anonymous)** can read all published content and send a contact message. The **Admin User (authenticated)** can manage content modules according to the assigned role. The **System Administrator (Super Admin)** is an Admin User who also has access to user management, role management, system settings, and activity logs; in the application this is the role named "Super Admin".

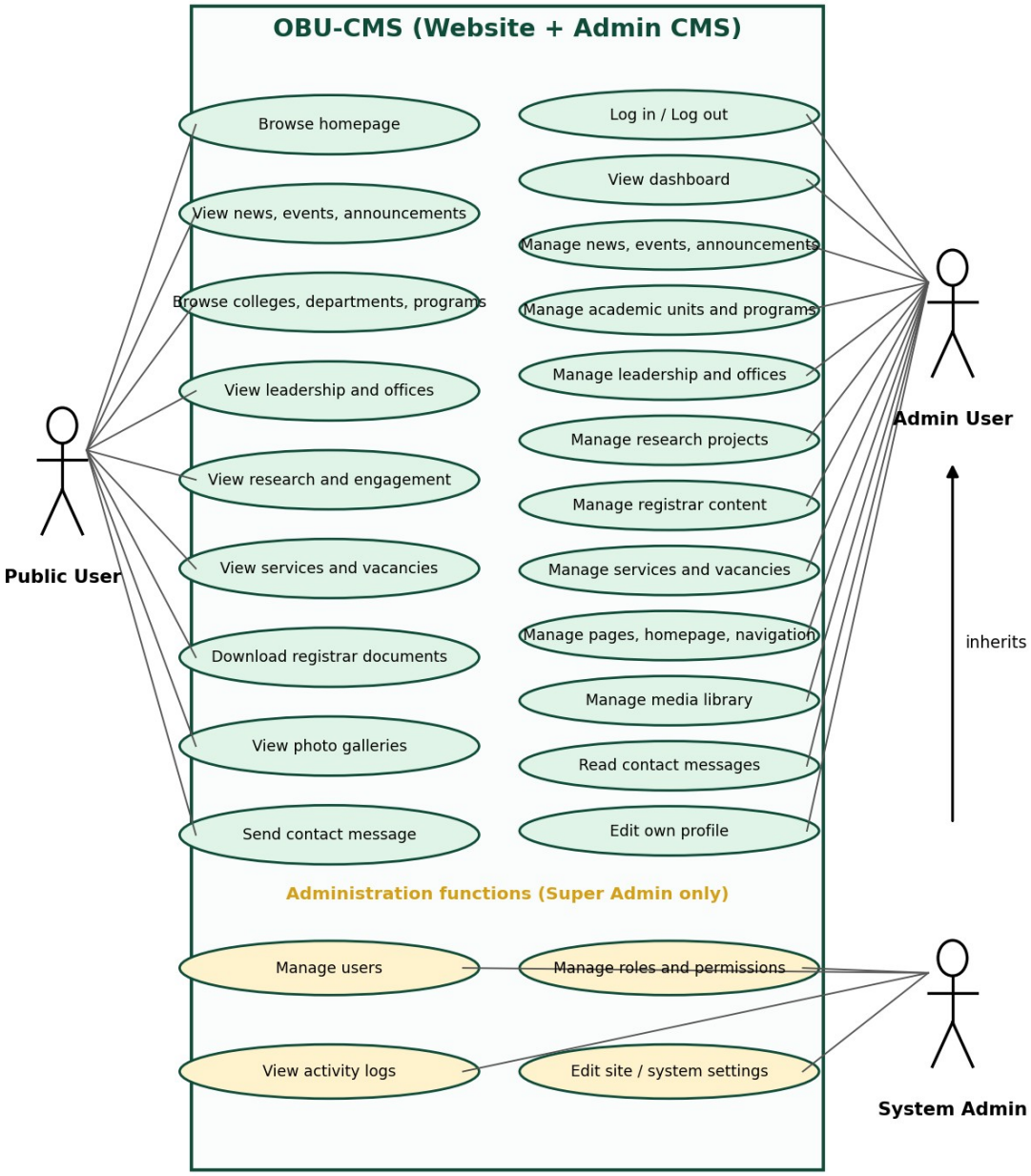


Figure 4.1: Use case diagram of OBU-CMS

Table 4.1: Primary use cases and actors

Actor	Main use cases	Pre-condition
Public User	Browse home page; read news, events, announcements; browse colleges, departments, programs; view leadership, offices, research, services, vacancies; download Registrar documents; view galleries; send a contact message	None
Admin User	Log in and out; view dashboard; manage news, events, announcements, academic units and programs, leadership, offices, research, Registrar content, services, vacancies, pages, homepage, navigation, and media; read contact messages; edit own profile	Valid account in Supabase Auth and in the users table
System Administrator	All Admin User use cases plus manage users, manage roles and permissions, view activity logs, edit site and system settings	User role is "Super Admin"

4.1.2. Use Case Description: Publish a News Article

Table 4.2: Use case description

Item	Description
Use case	UC-07 Publish a news article
Actor	Admin User (Content Editor)
Pre-conditions	The administrator is logged in and opens Content > News > Add.
Main flow	1. Enter title, category, short description, and content. 2. Optionally choose a cover image (uploaded to the media library). 3. Choose status Published. 4. Click Save. 5. The system makes the slug unique, stores the article with the author's name, writes an activity log entry, refreshes the list, and shows a success message.
Alternative flows	A1. Required field missing: the form shows an error and nothing is saved. A2. Image type or size not allowed: the upload is rejected with a message. A3. Session expired: the user is redirected to the login page. A4. Status Draft: the article is saved but stays hidden from the public.
Post-conditions	The published article is visible at /news/{slug} and listed on the news page; the action is recorded in activity_logs.

4.1.3. Class Diagram

Two class diagrams describe the system. Figure 4.2 shows the **core domain classes** (user, role, media, activity log, and the main content entities) with their attributes, methods, and relationships. The attributes are the real columns of the database tables. Figure 4.3 shows the **application modules** of the TypeScript code (context provider, hooks, routes, components) and how they depend on each other.

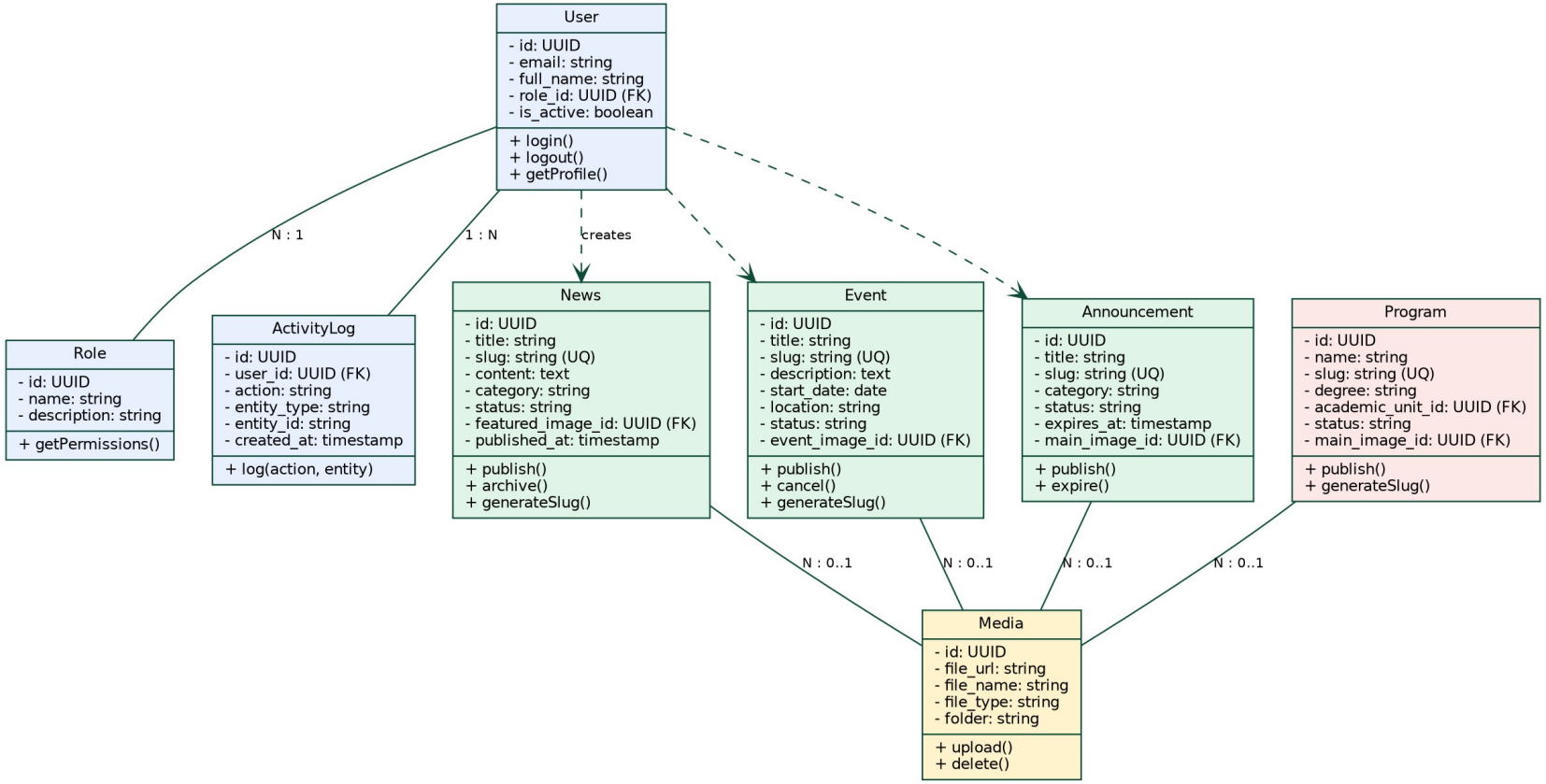


Figure 4.2: Class diagram of the OBU-CMS core domain

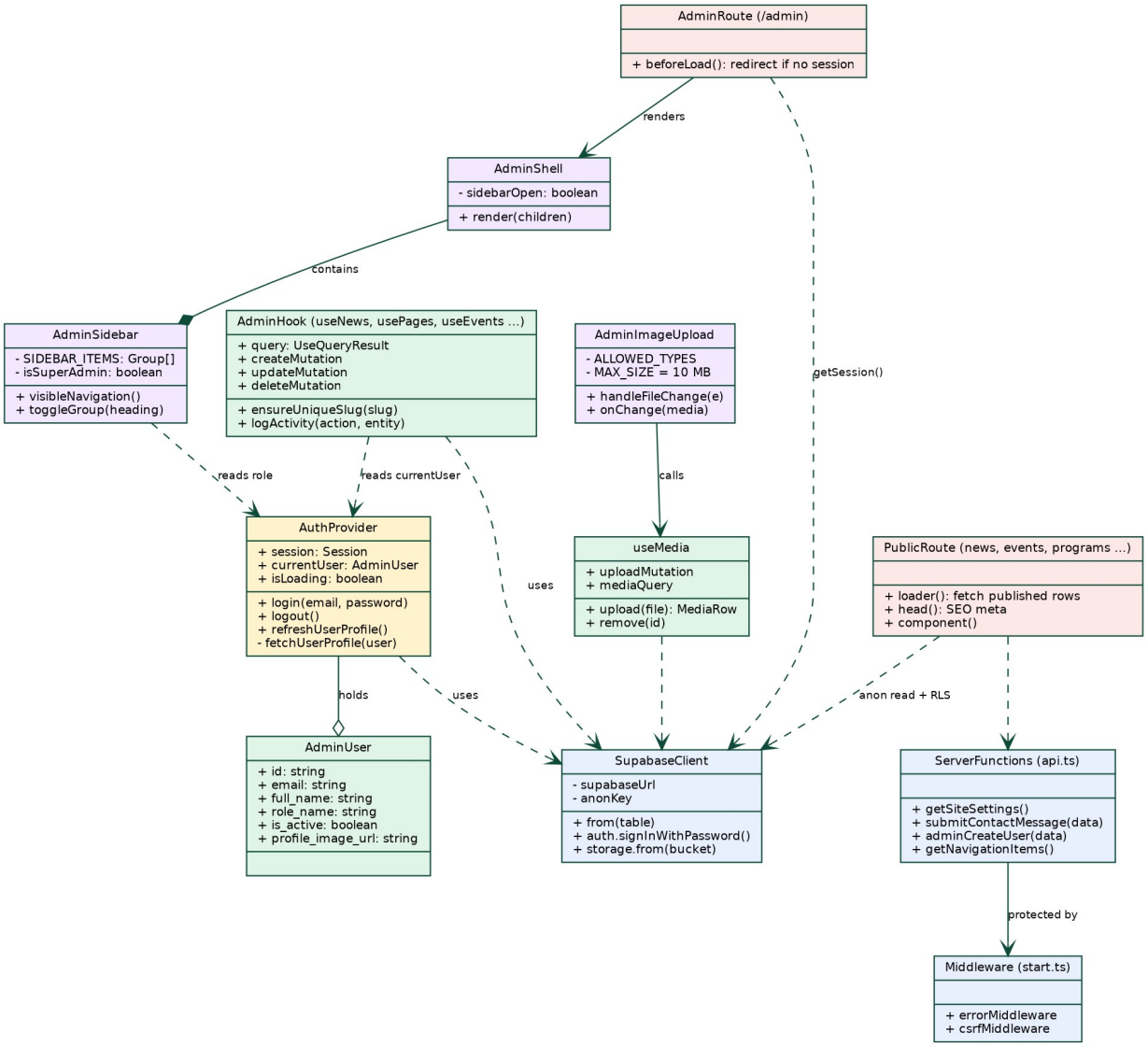


Figure 4.3: Class diagram of the main application modules

4.1.4. Object Diagram

The object diagram shows a snapshot of the system when a content editor publishes a news article: the instances, their values, and their runtime relationships.

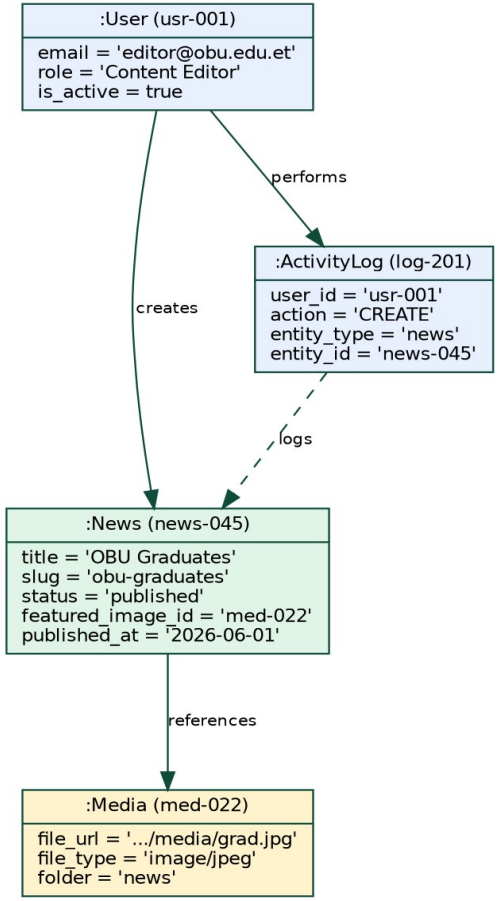


Figure 4.4: Object diagram - publishing a news article

4.1.5. Sequence Diagrams

Three sequence diagrams describe the most important interactions: administrator login with route protection (Figure 4.5), creating and publishing a news article (Figure 4.6), and a visitor reading an article (Figure 4.7).

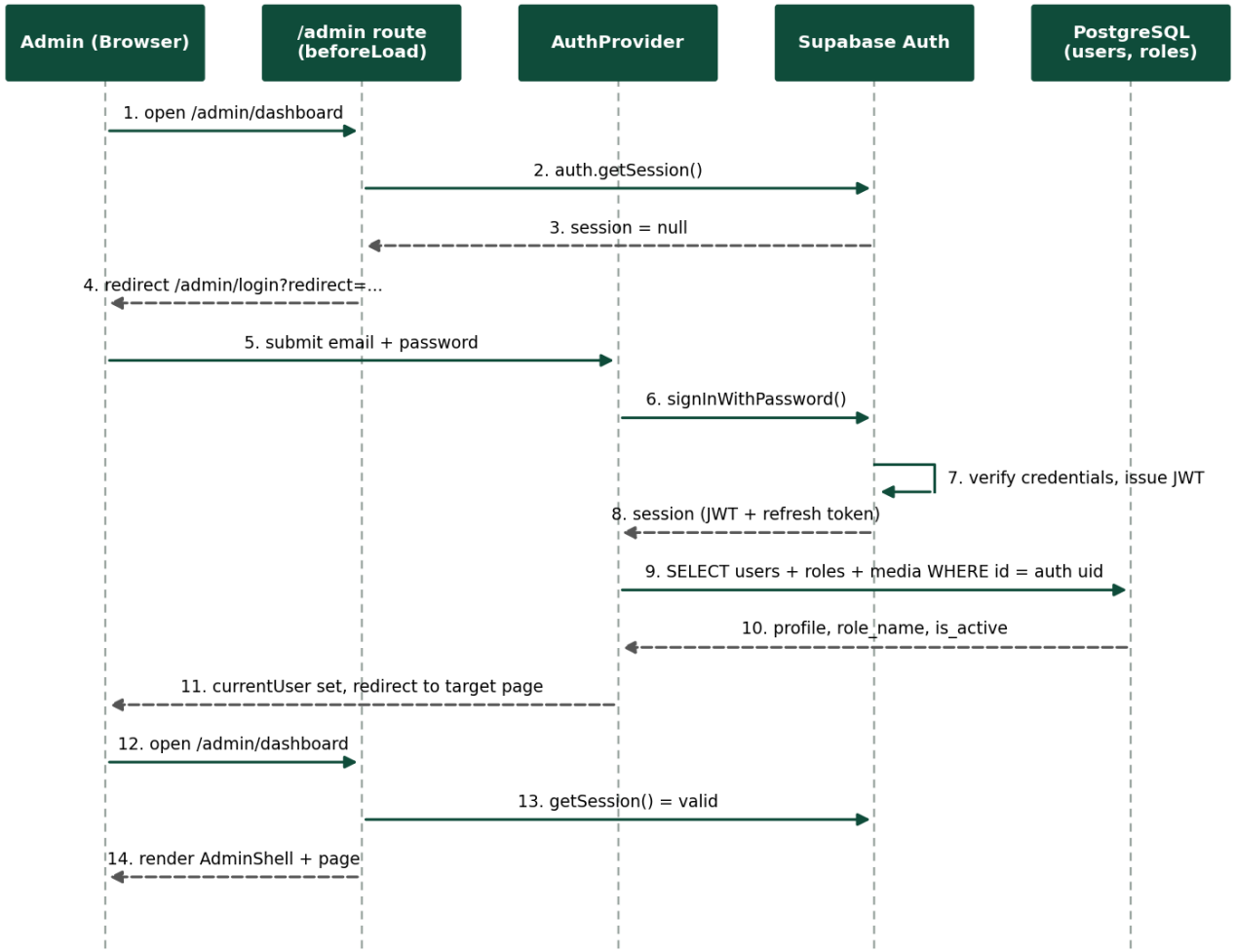


Figure 4.5: Sequence diagram - administrator login and route protection

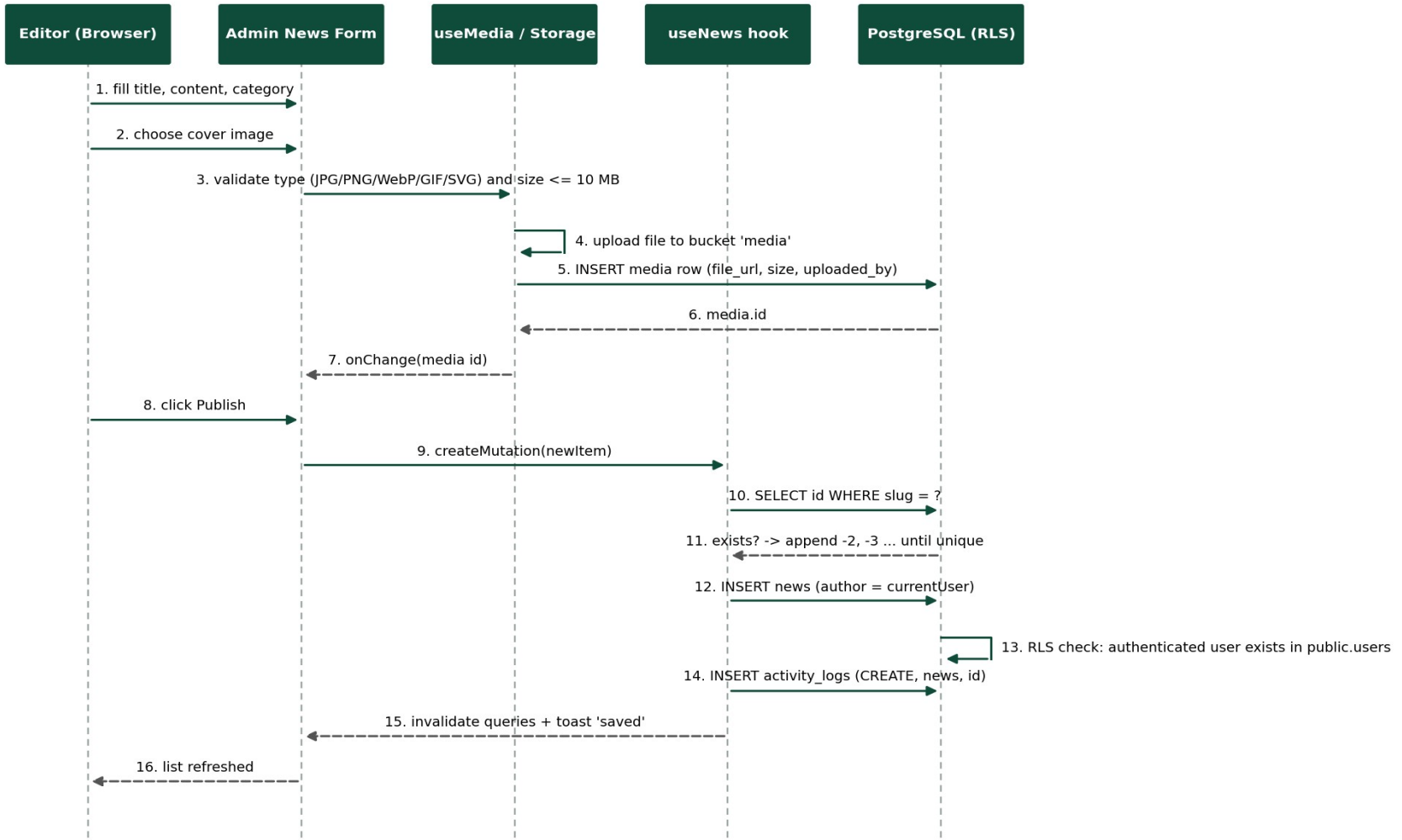


Figure 4.6: Sequence diagram - create and publish a news article with image upload

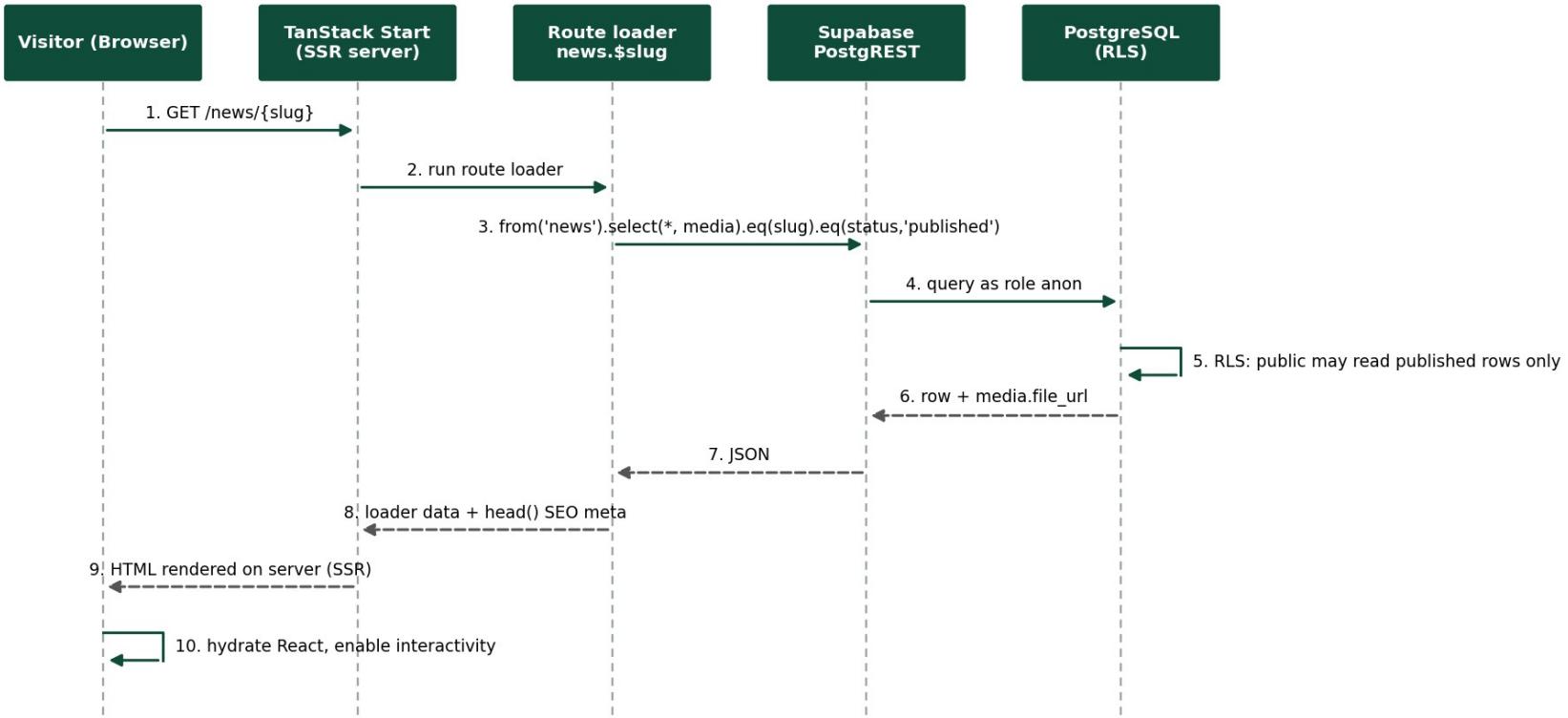


Figure 4.7: Sequence diagram - a visitor reads a published news article

4.1.6. State Chart Diagram

The state chart describes the lifecycle of a content item (news, announcement, event) from creation to deletion. In the current implementation an item is either **Draft** or **Published** (or Active / Inactive for modules with that field), announcements and events also expire by date, and administrators can delete items. The **Review** step before publishing is shown as a planned state, as recommended in Chapter 6.

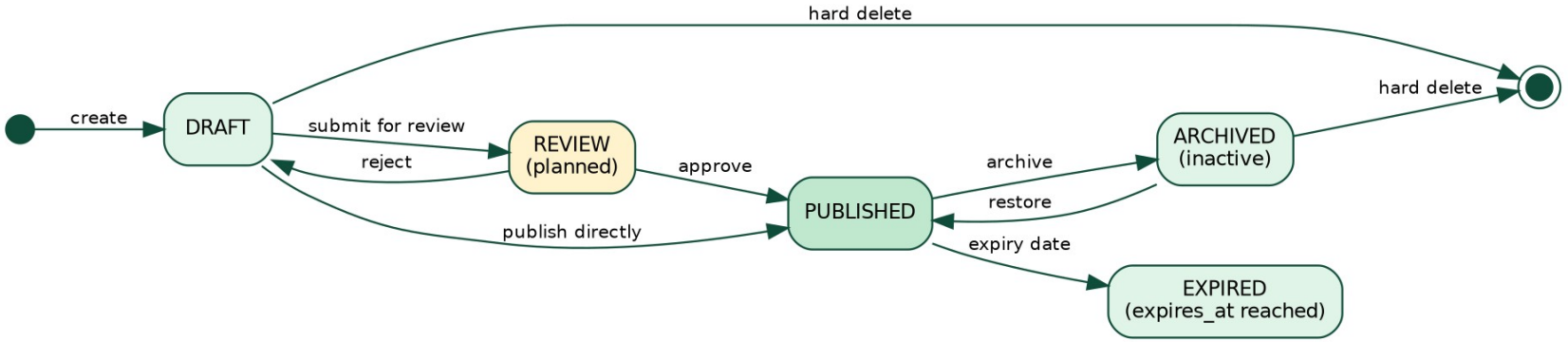


Figure 4.8: State chart diagram - content item lifecycle

4.1.7. Activity Diagram

Figure 4.9 shows the content publishing workflow common to all modules: session check, form validation, optional media upload, slug uniqueness check, saving with the Row Level Security check, activity logging, and the effect of the status on public visibility.

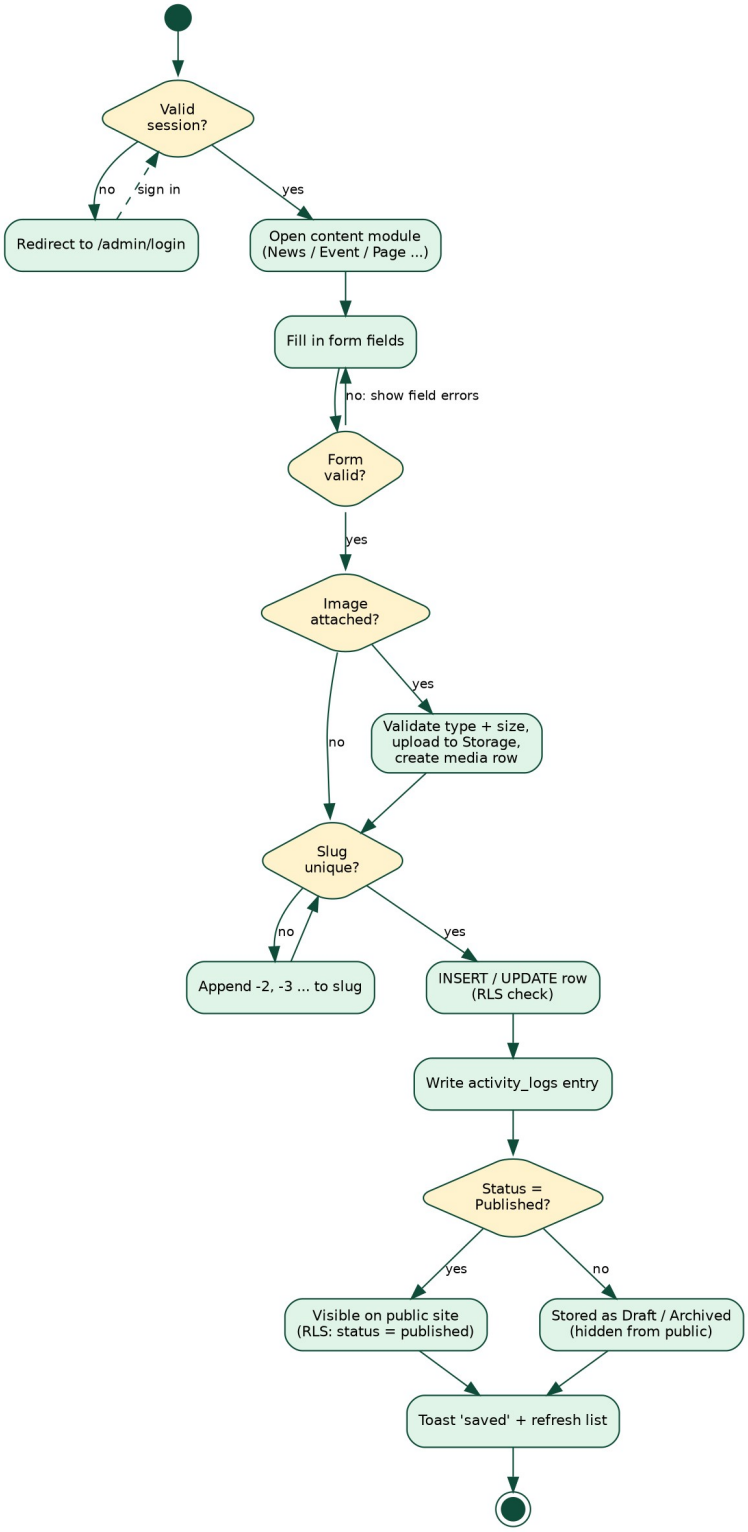


Figure 4.9: Activity diagram - admin content publishing workflow

4.2. User Interface Design

4.2.1. Navigational Flow

The navigational flow has two navigation trees: one for the public website (anonymous users) and one for the Admin CMS (authenticated users). The role of the logged-in user determines which sidebar sections are visible in the Admin CMS; the System group is shown only to the Super Admin.

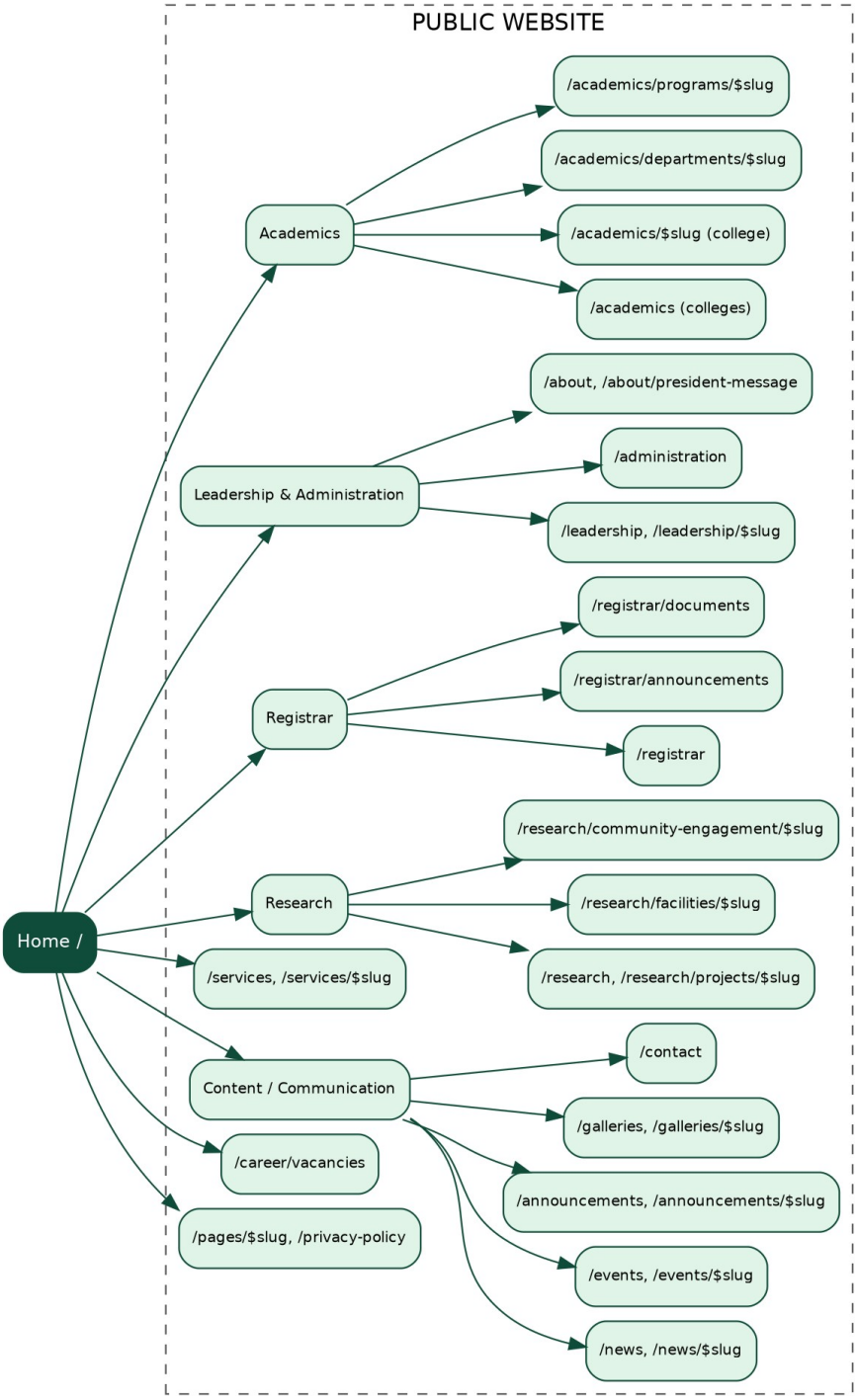


Figure 4.10: Navigational flow - public website (anonymous users)

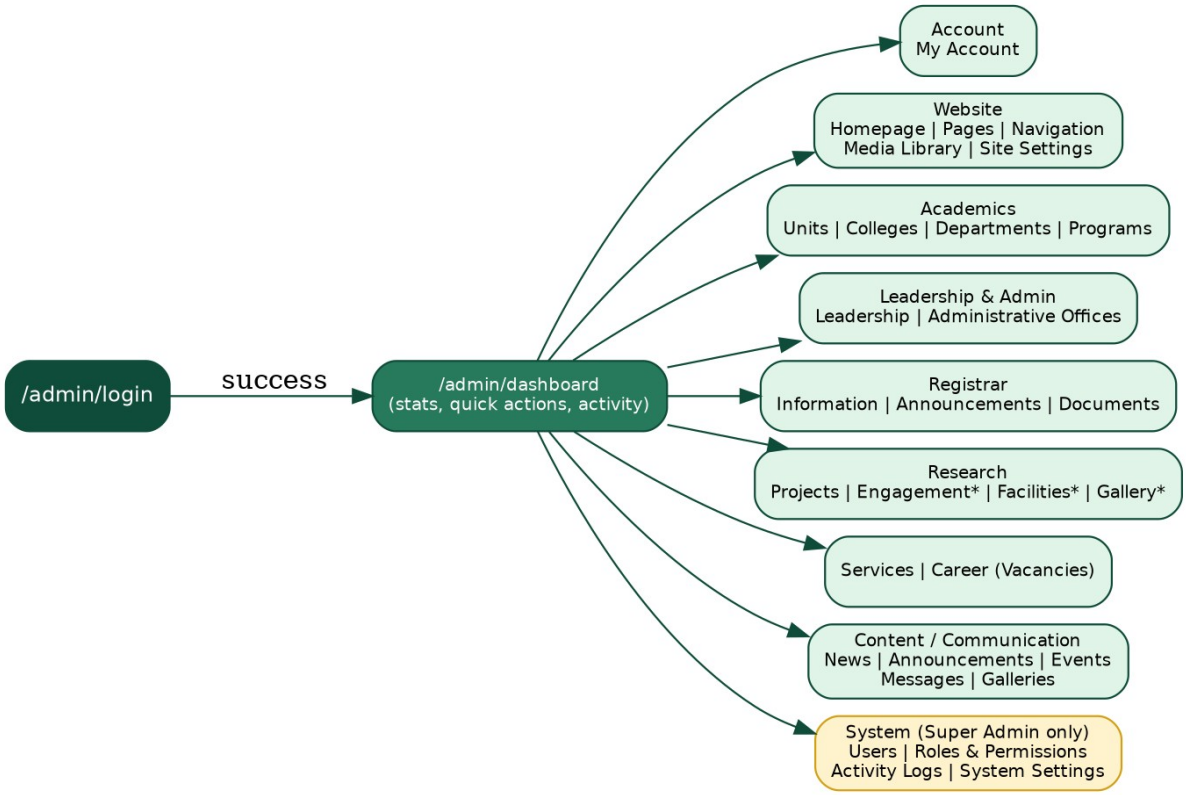


Figure 4.11: Navigational flow - Admin CMS (authenticated users); * = menu entry present, page still to be built

4.2.2. Key UI Design Decisions

- **University brand colors:** the primary color, a deep OBU green (#186a42), is used for action buttons, headings, and accents; the secondary color, gold (#ffcc00), is used for highlights, matching the visual identity of the university. The bilingual name (Afaan Oromo, Amharic, English) is shown in the header.
- **Admin CMS layout:** a fixed left sidebar with collapsible sections, a top header with search, the user profile and logout, and a main content area with a consistent card/panel layout; on small screens the sidebar becomes a drawer.
- **Data tables:** all admin list views use a standard table with image thumbnail, title, category, status badge, date, and action buttons (Edit, Delete).
- **Form layout:** create and edit forms use the same structure in every module, with the main content fields first and the metadata (status, category, featured image) beside or below them.
- **Status badges:** color-coded badges, for example green for published/active and yellow or grey for draft/inactive.
- **Toast notifications:** the Sonner library shows non-blocking success and error messages after every save, delete, or upload.
- **Accessibility and SEO:** accessible dialog, select, tab, and menu components from Radix UI, and descriptive page titles and meta descriptions on each public route.

4.2.3. Screens of the New System

The following screenshots, taken from the running system, show the main screens. The public site shows the hero slider, the President's message section, and a footer with links grouped as Essentials, About Us, Resources, and Quick Links. The Admin CMS shows the login page, the dashboard, the homepage manager, and the form for creating a user.

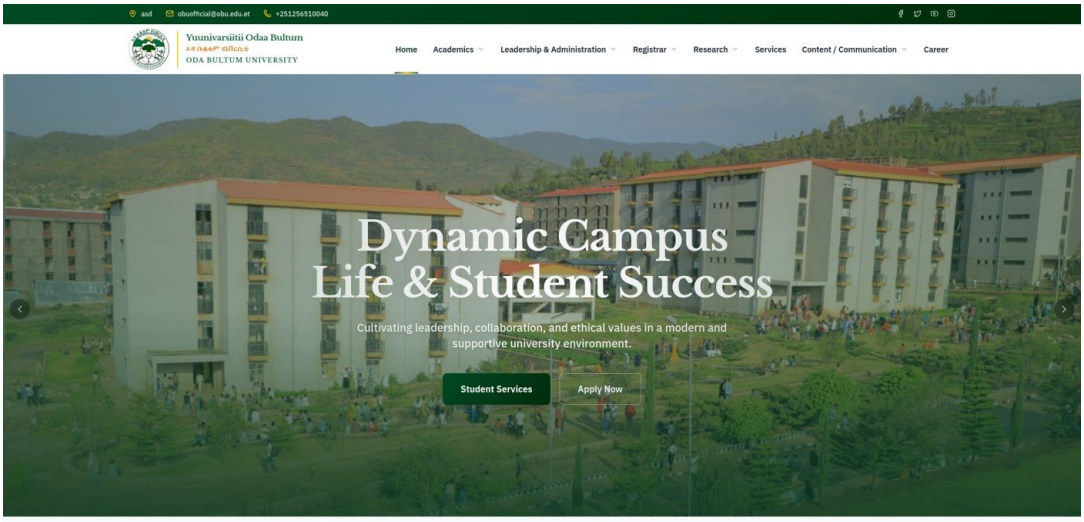


Figure 4.12: After - new public website: dynamic home page with hero slider and main menu

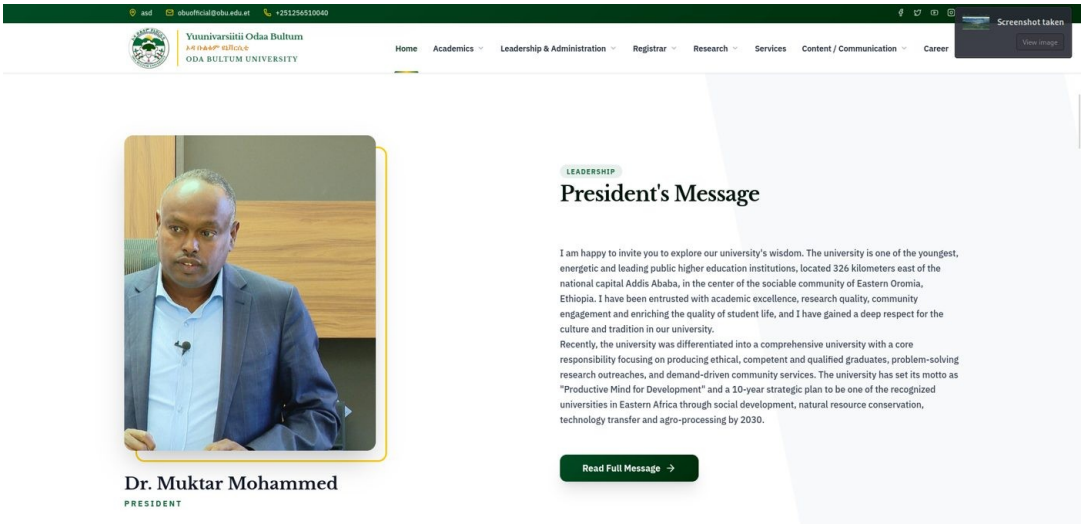


Figure 4.13: After - new public website: President's message section with photo and biography

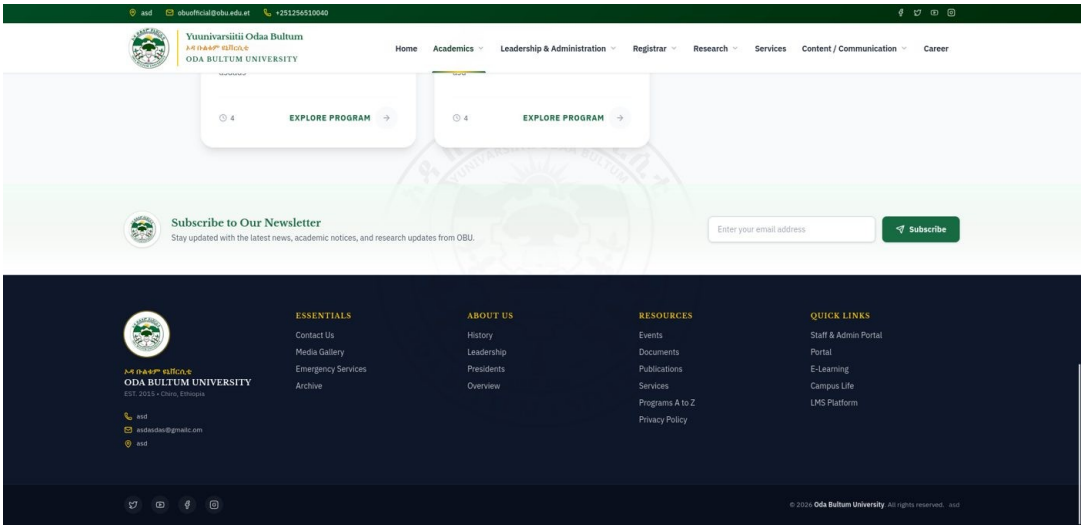


Figure 4.14: After - new public website: newsletter block and footer navigation

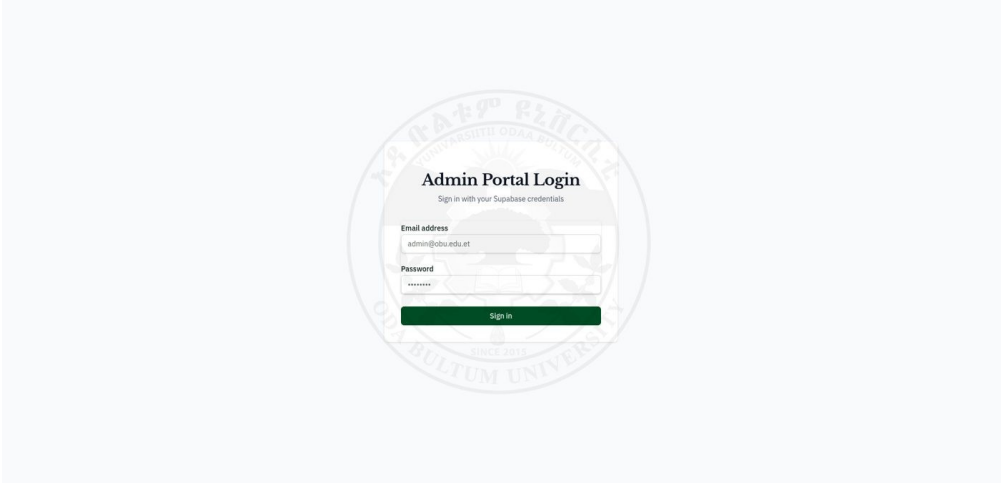


Figure 4.15: After - Admin CMS: Admin Portal Login (Supabase Auth, JWT session)

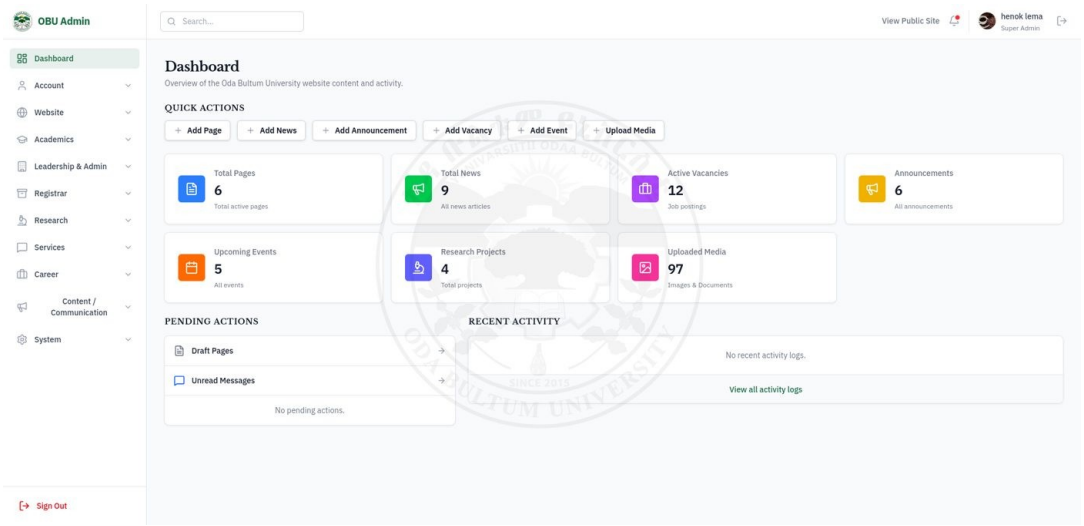


Figure 4.16: After - Admin CMS: dashboard with live content statistics and quick actions

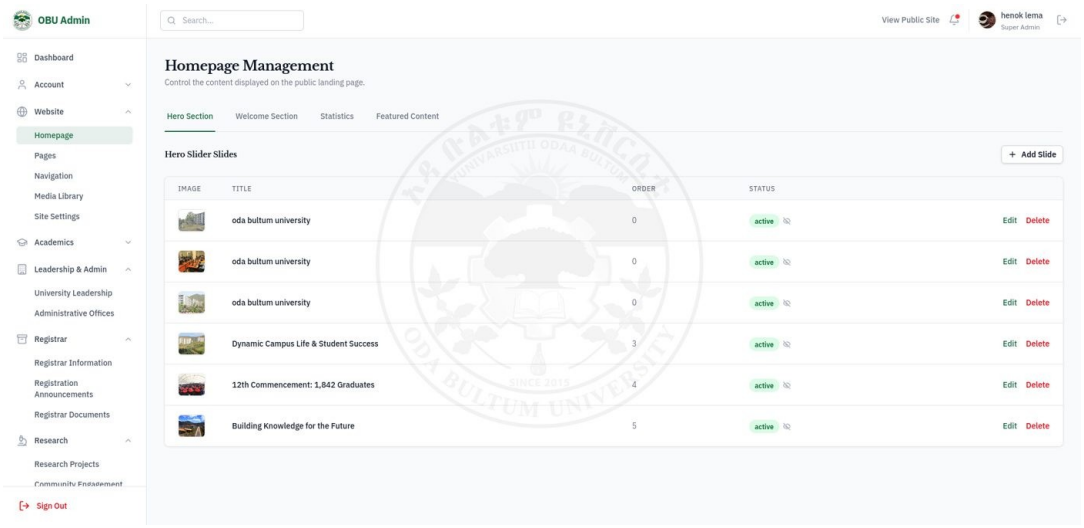


Figure 4.17: After - Admin CMS: homepage management (hero slides with status control)

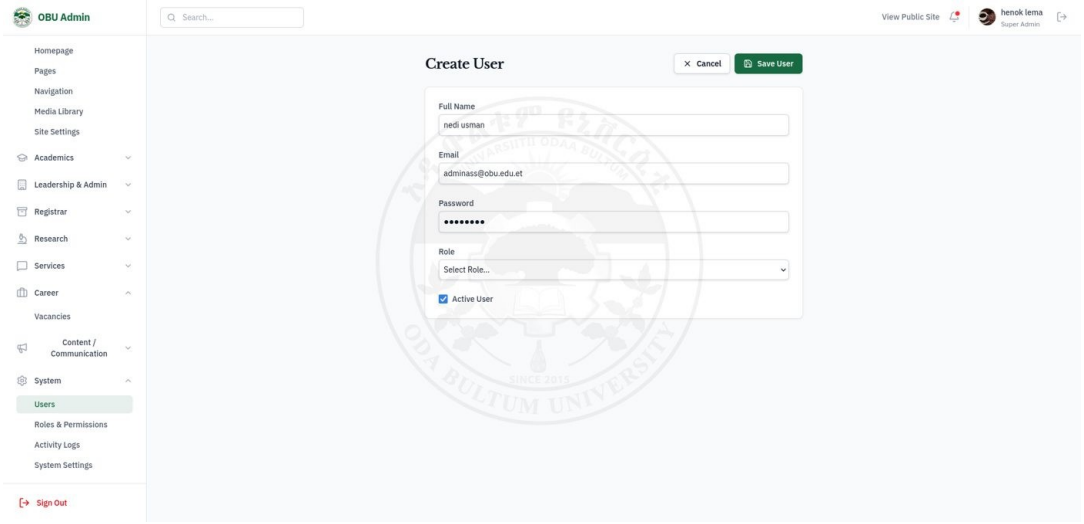


Figure 4.18: After - Admin CMS: create user form with role assignment

4.3. Software Architecture

4.3.1. Three-Tier Architecture

OBU-CMS follows a three-tier architecture that separates presentation, application logic, and data persistence (Figure 4.19).

- **Tier 1 - Presentation:** TanStack Router and React 19 (server-side rendering through TanStack Start); the public website routes and the Admin CMS routes; Tailwind CSS v4, Radix UI, and Lucide icons; TanStack Query for the client-side cache.
- **Tier 2 - Application:** the TanStack Start server (Node.js with Vite) with route loaders and 56 server functions (`createServerFn`) in `src/backend/api.ts`; CSRF and error middleware in `src/start.ts`; the data hooks of the Admin CMS; and an event endpoint (`/api/events`) for live refresh.
- **Tier 3 - Data:** the Supabase cloud project: PostgreSQL with 27 tables and Row Level Security, Supabase Auth (JWT sessions), Supabase Storage for media files, and the automatic REST interface (PostgREST).

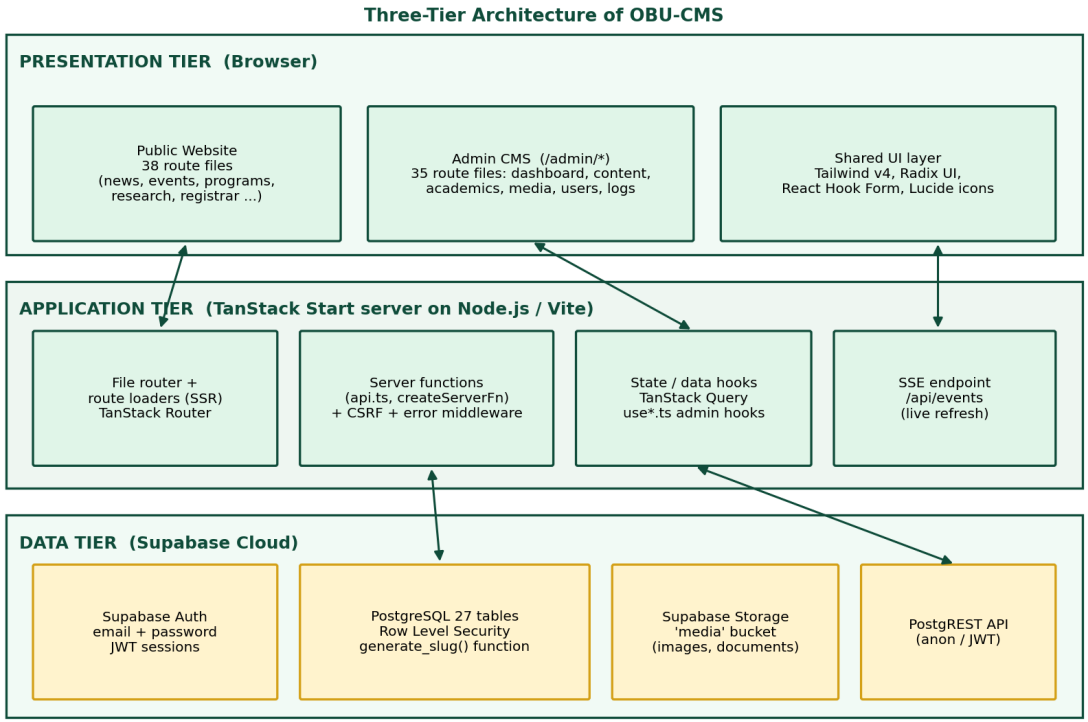


Figure 4.19: Three-tier software architecture of OBU-CMS

4.3.2. Component Diagram and Subsystem Decomposition

Figure 4.20 shows the major software components and their dependencies. Public routes mostly read through the Supabase client with the anonymous key; Admin routes use the authentication context and the data hooks; server functions use a direct PostgreSQL pool or the Supabase service client. Table 4.3 lists the subsystems with their responsibilities and locations.

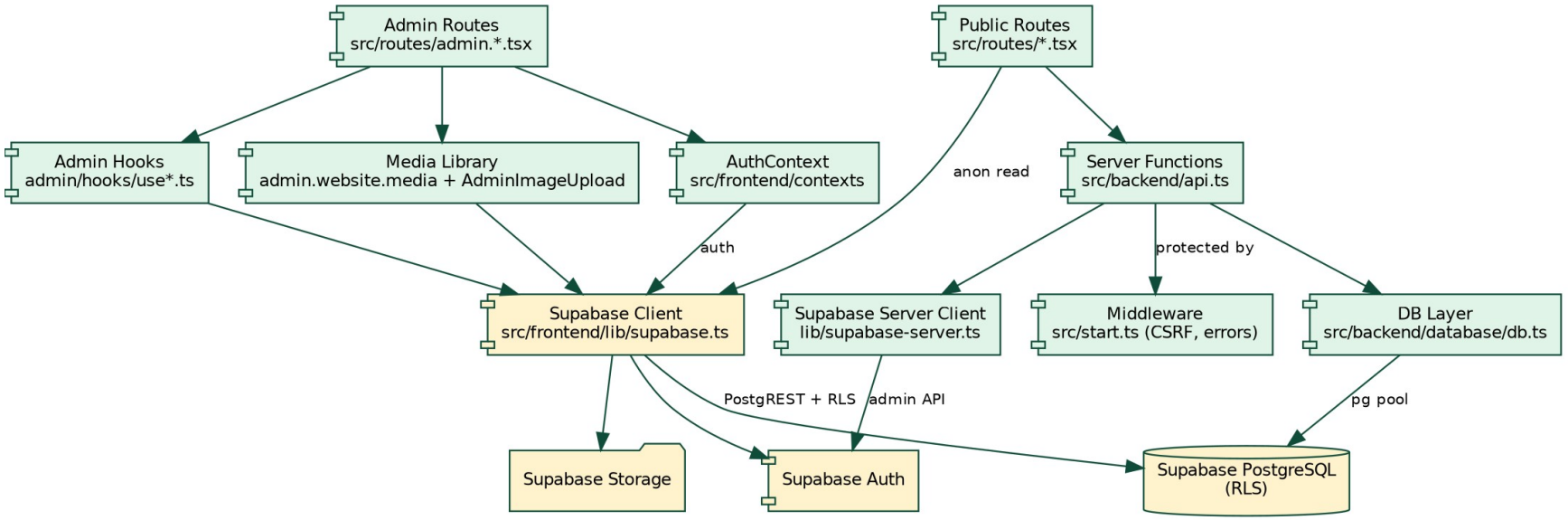


Figure 4.20: Component diagram of OBU-CMS

Table 4.3: Subsystems of OBU-CMS

Subsystem	Responsibility	Main location
Public website	Pages for visitors: home, news, events, academics, research, registrar, services, vacancies, galleries, contact	src/routes/*.tsx (38 public files), src/frontend/components
Admin CMS	Protected area /admin: dashboard, content modules, media, users, roles, logs, settings	src/routes/admin.*.tsx (35 files), src/frontend/admin
Authentication and session	Login, logout, current user profile and role	src/frontend/contexts/ AuthContext.tsx, route guard in src/routes/admin.tsx
Data access	Typed queries and mutations per module with cache refresh and activity logging	src/frontend/admin/hooks/ use*.ts (22 hooks), route loaders
Server functions	Server-side operations: site settings, navigation, contact messages, user creation, some listings	src/backend/api.ts, src/backend/lib/supabase-server.ts, src/start.ts

Subsystem	Responsibility	Main location
Media management	Upload, validation, storage, and reuse of images and documents	AdminImageUpload.tsx, useMedia.ts, Storage bucket media
Database and policies	27 tables, relations, slug function, access rules	supabase/migrations, src/frontend/lib/database.types.ts

4.3.3. Deployment Diagram

Figure 4.21 shows the deployment. Users reach the application over HTTPS. The application is built with Vite into a Node.js server entry (server.ts). Browsers also talk directly to Supabase with the public (anonymous) key or the signed-in user's token, and the access rules in the database decide what each request may read or write. Secrets such as the service key and the database connection string stay in the server's environment file. The screenshots in this report were captured on the development server (port 5173) connected to the Supabase project.

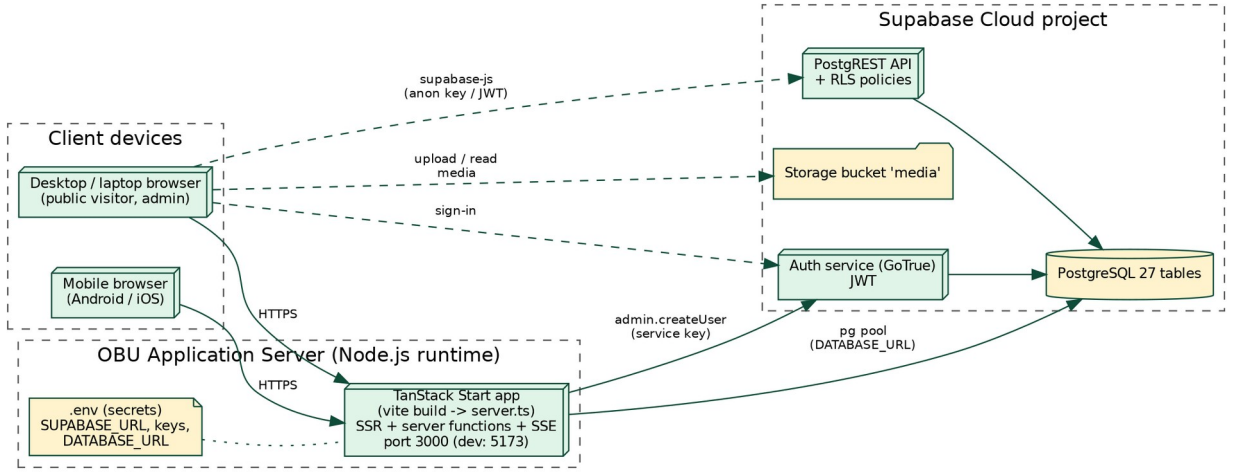


Figure 4.21: Deployment diagram of OBU-CMS

4.4. Security and Access Control

4.4.1. Authentication

Authentication uses **Supabase Auth** with email and password. The flow is: (1) the administrator submits credentials on /admin/login; (2) supabase.auth.signInWithPassword() checks them against a salted hash stored by Supabase and returns a signed JSON Web Token (JWT) session; (3) the Supabase SDK stores the session in the browser; (4) AuthContext listens to onAuthStateChange() to keep the session in sync; (5) the administrator's profile, including the role name, is loaded from public.users; (6) every /admin/* route runs a TanStack Router beforeLoad guard that redirects users without a session to /admin/login.

The application never stores or handles password hashes. When a user is created, the server function adminCreateUser creates the account in Supabase Auth with the service key and then inserts a profile row in users with the same id; the password_hash column of that table holds only the marker text SUPABASE_AUTH_DELEGATED.

4.4.2. Authorization (RBAC)

Role-Based Access Control is applied at three levels:

- **Application level:** the Admin sidebar hides the whole **System** group (Users, Roles and Permissions, Activity Logs, System Settings) unless the user's role is "Super Admin".
- **Database level (RLS):** PostgreSQL Row Level Security enforces access at the data layer. Public users can only SELECT rows with status = 'published' or 'active'. Write policies require an authenticated user who exists in public.users (Table 4.5). This means that even if the interface is bypassed, the database does not return unauthorized data.
- **Storage level:** the media bucket is publicly readable, while upload, update, and delete require an authenticated user listed in users.

Table 4.4 shows the intended access matrix for the four user groups of the university. The System Administrator restriction is enforced today; the separation between Content Editor and Registrar Staff per module is part of the design and is listed in Chapter 6 as a database-level improvement.

Table 4.4: RBAC access matrix - role permissions

Resource	Public User	Content Editor	Registrar Staff	System Admin
View published content	Yes	Yes	Yes	Yes
Manage news, events, announcements	No	Yes	Yes	Yes

Resource	Public User	Content Editor	Registrar Staff	System Admin
Manage Registrar content	No	No	Yes	Yes
Manage research and services	No	Yes	No	Yes
Upload media	No	Yes	Yes	Yes
Manage users and roles	No	No	No	Yes
View activity logs	No	No	No	Yes
Configure site settings	No	No	No	Yes

Table 4.5: Row Level Security policies written in the migration files

Object	Policy	Rule
pages	Public read active pages	SELECT where status = 'published'
pages	Admin write pages	ALL for authenticated users that exist in users
academic_units	Public read active units / Admin write units	SELECT where status = 'active'; ALL for authenticated users in users
programs	Public read active programs / Admin write programs	SELECT where status = 'active'; ALL for authenticated users in users
users	Users can update their own profile	RLS enabled; each user may update only their own row
storage.objects (bucket media)	Public media read / Admin media upload, update, delete	SELECT for everyone; INSERT, UPDATE, DELETE for authenticated users in users

The policies for the other content tables (news, events, announcements, and so on) were configured in the Supabase project and are inspected with the helper scripts check_rls.ts and query_policies.js. Recording them in migration files is a recommended improvement so that the database can be rebuilt exactly.

4.4.3. CSRF Protection

All server functions (createServerFn) are protected by createCsrfMiddleware, configured in src/start.ts and applied to server-function requests. This validates the origin of state-changing requests and prevents cross-site request forgery. An error middleware in the same file returns a friendly error page instead of stack traces.

4.4.4. Input Validation and SQL Injection Prevention

- Server-side SQL uses parameterized queries from node-postgres ($1, $2, ...), and the Supabase client builds parameterized requests; together they prevent SQL injection.
- Uploaded files are limited to JPG, PNG, WebP, GIF, and SVG images and to 10 MB by AdminImageUpload; unsupported files are rejected with a message.
- React escapes text by default, which reduces the risk of cross-site scripting; forms check required fields.
- Zod is installed in the project for schema validation of form and server-function inputs; applying it to every server function is a recommended improvement (Chapter 6).

4.4.5. Encryption and Data Protection

- **In transit:** the Supabase endpoints use HTTPS, and the website is intended to be served over HTTPS.
- **Passwords:** hashed and salted by Supabase Auth; never stored in plain text by the application.
- **Secrets:** the service-role key, database URL, and project URL are read from environment variables (`src/backend/.env`); only the public anonymous (publishable) key is used in the browser.

Table 4.6: Security controls and where they are enforced

Control	Where enforced	Layer
Login and session	Supabase Auth; `/admin` route guard	Authentication
Read only published or active rows	RLS policies	Database
Writes only by registered users	RLS policies; storage policies	Database
System menu for Super Admin	Sidebar role check	Application
Cross-site request protection	CSRF middleware	Server
Upload type and size limits	`AdminImageUpload`	Application
Audit trail	`activity_logs` written by data hooks	Application

4.5. Database Design

4.5.1. Entity Relationship Diagram (ERD)

The database has 27 tables. Figures 4.22 and 4.23 show them and their foreign keys. All content tables reference the central `media` table, and administrator actions are tied to `users` through `activity_logs`. Most pages read and write through the Supabase client (`supabase-js`); a smaller set of server functions uses a direct PostgreSQL connection for settings, navigation, contact messages, and creation of users.

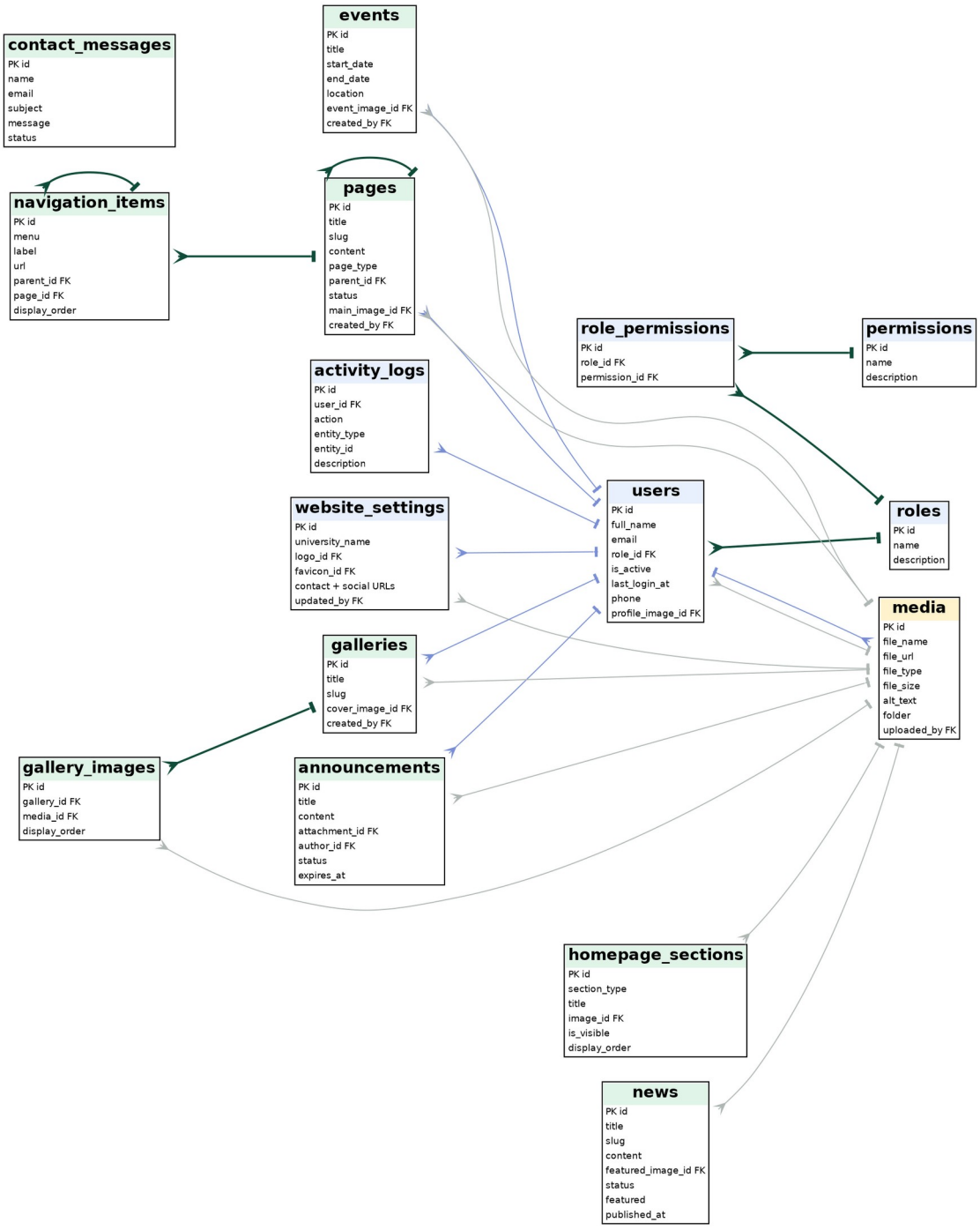


Figure 4.22: Entity-relationship diagram (1) - security, settings, and web content

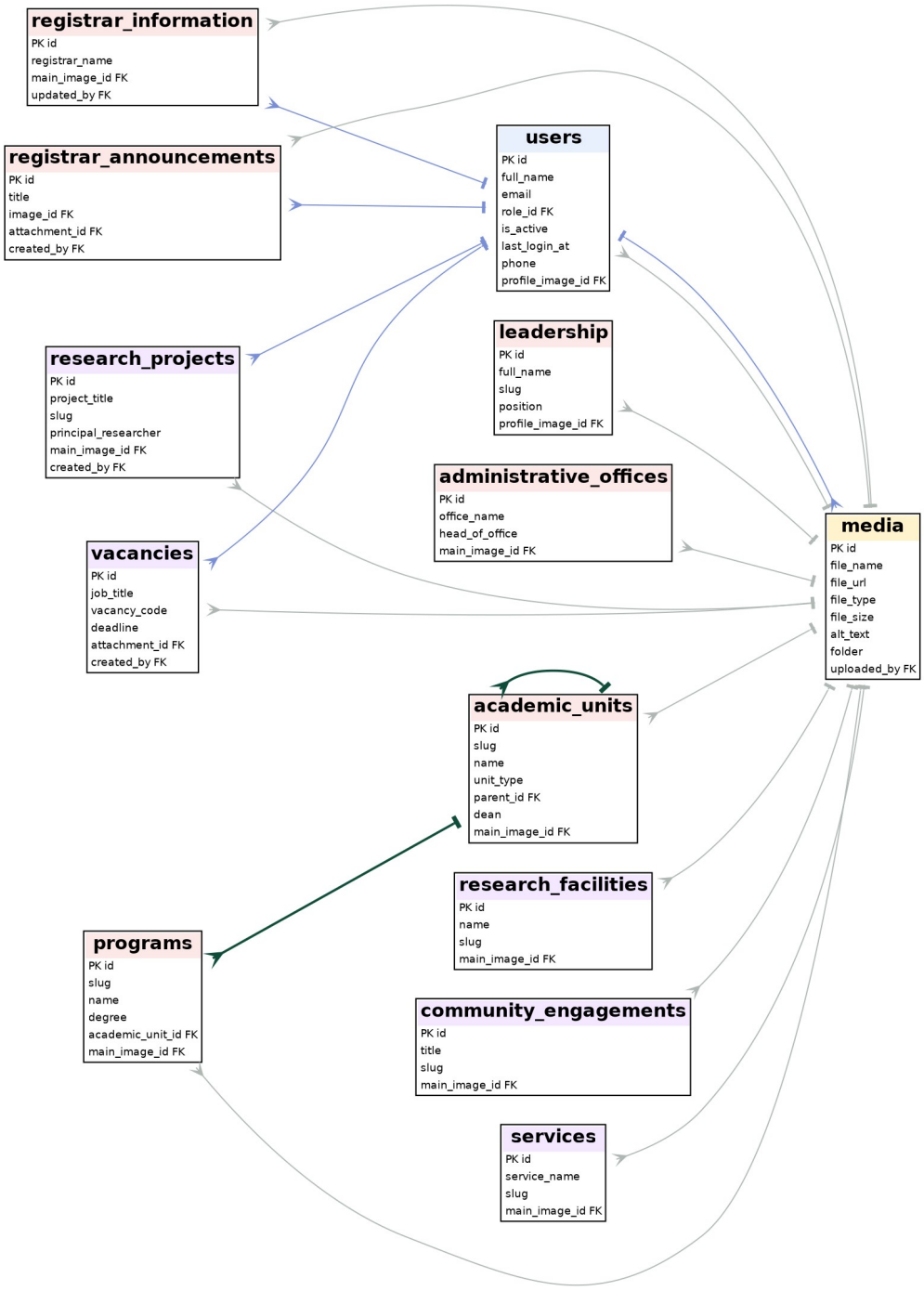


Figure 4.23: Entity-relationship diagram (2) - academics, administration, research, and services

In the diagrams, the crow's foot marks the "many" side. Dark green lines are relations between content tables; thin grey lines are references to media; blue lines are references to users. Some columns are omitted for readability.

Table 4.7: Database tables by module (27 tables)

Module	Tables	Purpose
Security and audit	users, roles, permissions, role_permissions, activity_logs	Staff accounts, roles with permissions, and the audit trail.

Module	Tables	Purpose
Site and web content	website_settings, pages, navigation_items, homepage_sections, media, contact_messages	Site identity and contacts, general pages, menus, home page blocks, uploaded files, messages from visitors.
Communication	news, announcements, events, galleries, gallery_images	News, notices, events, and photo galleries.
Academics	academic_units, programs	Colleges and departments (hierarchy) and degree programs.
Administration and Registrar	leadership, administrative_offices, registrar_information, registrar_announcements	University leaders, offices, and the Registrar's information and notices.
Research and services	research_projects, research_facilities, community_engagements, services, vacancies	Research, community engagement, services, and job vacancies.

4.5.2. Key Database Design Decisions

- **UUID primary keys:** tables use UUID primary keys generated by PostgreSQL, which avoids guessable sequential ids and supports distributed inserts.
- **Slug unique constraints:** content tables with public detail pages have a unique slug column, enforced by the database to prevent duplicate URLs.
- **Centralized media table:** one media table stores all file metadata; content tables store a foreign key (main_image_id, featured_image_id, attachment_id, profile_image_id) so media can be reused across entities.
- **Status-based visibility:** visibility is controlled by a status column (draft / published, active / inactive) instead of hard deletion, and public queries and policies select only visible rows.
- **Hierarchies with self reference:** academic_units.parent_id (college to department), pages.parent_id, and navigation_items.parent_id; programs.academic_unit_id links a program to its unit.
- **Role model:** users.role_id references roles; role_permissions joins roles and permissions (many-to-many).
- **Audit log on every change:** every create, update, delete, and upload action made through the data hooks inserts a row in activity_logs.
- **Security in migrations:** Row Level Security and its policies are created in migration files, so security is part of the deployment from the beginning (for the tables covered by the migrations).

5

CHAPTER 5
Implementation

Development process, tools used, challenges, and testing results

Sections in this Chapter

- 5.1 Development Process
- 5.2 Tools Used
- 5.3 Challenges and Solutions
- 5.4 Testing

CHAPTER 5: IMPLEMENTATION

This chapter documents the development process of the OBU-CMS project, the tools and technologies used, the technical challenges encountered and how they were resolved, and the results of the testing activities.

5.1. Development Process

Development followed the Agile Scrum sprint structure defined in Chapter 2. The key implementation work of each sprint was:

- **Sprint 1 - Authentication and database foundation:** the Supabase project was created and the PostgreSQL schema was deployed through migration files in `supabase/migrations/`. Row Level Security policies were configured. The `AuthContext` was implemented to hold the session state of the whole application. The Admin CMS skeleton (sidebar, header, layout) was built and the admin login page was connected to Supabase Auth.
- **Sprint 2 - Public website:** all public routes were implemented with TanStack Router file-based routing. Route loaders read the published data, and server functions in `src/backend/api.ts` serve settings, navigation, and the contact form. The home page was built with hero slides, latest news, upcoming events, research highlights, and site statistics. Dynamic slug routes were implemented for news, events, announcements, programs, departments, leadership, and research pages.
- **Sprint 3 - Admin CMS core modules:** admin CRUD panels were built for News, Events, Announcements, Programs, Academic Units, Leadership, Services, and Vacancies. The `AdminImageUpload` component was built on top of Supabase Storage, and the Media Library admin page, navigation management, and website settings panels were implemented.
- **Sprint 4 - Extended modules:** admin panels were added for Research Projects, Administrative Offices, Registrar information, announcements, and documents. The activity log was implemented: the data hooks write every create, update, delete, and upload action to `activity_logs`. The dashboard was finished with live counts from the content tables and the recent activity feed. The tables and public pages for research facilities and community engagement exist; their admin panels are still to be built.

5.1.1. Project Structure

Table 5.1: Project folder structure

Folder / file	Purpose
src/routes/	76 entries: 38 public page files, 35 admin page files, the __root.tsx layout, the api/events.ts endpoint, and a README. File names define the address, for example news.$slug.tsx is /news/{slug}.
src/frontend/components/	Public interface components: Navbar, Footer, PageHero, cards, carousel, breadcrumbs.
src/frontend/admin/	Admin layout (AdminShell), components (sidebar, topbar, page header, badge, image upload), and 22 data hooks (useNews, usePages, useMedia, useUsers, ...).
src/frontend/contexts/	AuthContext with login, logout, and the current user.
src/frontend/lib/	Supabase client, generated and extended database types, error page and error capture.
src/backend/	api.ts (server functions), database/db.ts (PostgreSQL pool), lib/supabase-server.ts (service-key client), events.ts (event channel), server.ts (server entry).
src/start.ts	Global middleware: error handling and CSRF protection.
supabase/migrations/	Ordered SQL files that change the database (Table 5.2).
specs/university-admin-media/	Requirements, design, and task documents.
document/	Screenshots of the old and new websites and report material.
Root scripts (check_rls.ts, inspect_*.ts, test_*.sh, seed-admin.ts)	Helper scripts to inspect the database and policies, check pages with curl, and create a development admin.

5.1.2. How a Module Is Implemented

Every Admin CMS module follows the same pattern, which made the work repeatable:

1. **Table and migration:** create or extend the table in a migration file and update the typed definitions.
2. **Data hook:** write use<Module>.ts with a query (list with joined media) and create, update, and delete mutations that also write to activity_logs and show a toast.
3. **Admin screens:** a list page with search and status badges, and a form page with AdminImageUpload for images.
4. **Public page:** a route with a loader that reads only published or active rows, a head() function for the page title and description, and a slug-based detail route.
5. **Policies and sidebar entry:** add the Row Level Security policy and the menu item.

For example, the news hook lists articles ordered by publication date with the cover image joined from media, makes the slug unique by adding a number, sets the author from the signed-in user, and records "Created news article" in the activity log (Appendix A.3).

5.1.3. Database Migration History

Table 5.2: Migration history

Migration file	Change
20260825_phase2a_schema.sql	Slug columns for galleries, leadership, services, academic units, programs, and offices; the generate_slug() function; back-filling of existing rows; unique constraints.
20260825_phase2b_rls.sql	Row Level Security and policies for pages, academic_units, and programs.
20260825_media_bucket.sql	Public media storage bucket and its four storage policies.
20260825_website_title.sql	Website title field in the site settings.
20260831110000_add_research_projects_slug.sql	Slug and unique index for research projects.
20260903000000_add_announcements_main_image_id.sql	Cover image column for announcements.
20260903010000_add_slug_to_announcements_and_events.sql	Slugs and unique constraints for announcements and events.
20260903020000_add_profile_fields_to_users.sql	Phone and profile image for users; policy so users can update their own profile.
20260922000000_footer_navigation_seed.sql	Footer navigation items stored in navigation_items.

5.2. Tools Used

Table 5.3: Development tools by category

Category	Tool	Version	Purpose
Framework	TanStack Start	1.168.x	Full-stack SSR framework and server functions
Routing	TanStack Router	1.170.x	File-based type-safe routing
State management	TanStack Query	5.101.x	Server state, caching, background sync
Language	TypeScript	5.8.x	Static typing across the codebase
UI library	React	19.2	User interface components
Styling	Tailwind CSS	4.2.x	Utility-first responsive CSS
UI primitives	Radix UI	latest	Accessible modal, dropdown, dialog components
Forms	React Hook Form	7.71.x	Form state management
Validation	Zod	3.24.x	TypeScript-first schema validation (installed)
Database	Supabase PostgreSQL	Cloud	Cloud-hosted DB with RLS and Auth
DB client	node-postgres (pg)	8.23.x	Direct parameterized SQL queries
Supabase client	supabase-js	2.112.x	Auth, database access, storage from the app
File storage	Supabase Storage	Cloud	Media file upload and delivery
Build tool	Vite	8.1.x	Development server and production build
Icons	Lucide React	0.575.x	SVG icon set
Notifications	Sonner	2.0.x	Toast notification system
Version control	Git / GitHub	-	Source code management

Category	Tool	Version	Purpose
IDE	VS Code	latest	Development environment
API testing	Postman, curl scripts	latest	Endpoint testing and validation
Linting	ESLint + typescript-eslint	9.32.x	Code quality enforcement
Formatting	Prettier	3.7.x	Code style consistency

5.3. Challenges and Solutions

Challenge 1: Access control for mixed public and admin use

The most complex technical challenge was giving anonymous visitors read access to published content while giving authenticated administrators write access. The difficulty was that server functions that use a direct PostgreSQL connection (node-postgres) bypass Row Level Security, while queries made with the Supabase client respect it.

Solution: Row Level Security became the main protection. Public pages and admin hooks use the Supabase client, so the policies decide what is visible and writable: the public key can only read published or active rows, and writes need an authenticated user listed in users. Direct-connection server functions were kept for a small set of operations (settings, navigation, contact messages, and user creation) and are protected with CSRF middleware. Additional policies were added through migrations for programs, pages, academic_units, and the media bucket.

Challenge 2: Slug generation and uniqueness across tables

Many entities needed unique URL slugs generated from their names while handling duplicate names, special characters, and non-Latin text such as Amharic.

Solution: a PostgreSQL function generate_slug(base_text) was created in the Phase 2A migration. It lowercases the text, replaces non-alphanumeric characters with hyphens, and trims the edge hyphens. Existing duplicates were back-filled by appending the first 8 characters of the row's UUID, and unique constraints were then added to each slug column. For new records, the data hooks check the database and add -2, -3, ... until the slug is unique. Titles written only in Amharic (Ge'ez) script produce the generic slug "item", so a manual slug or transliteration is recommended.

Challenge 3: Media reuse across content entities

Different content types (news, events, programs, leadership, research) all needed images. Storing image URLs as text in each table created duplication and made central media management impossible.

Solution: a central media table became the single source of truth. Each content table has a foreign key (main_image_id, featured_image_id, and so on) referencing media.id. The

AdminImageUpload component handles the whole upload-and-attach workflow: it uploads to Supabase Storage, creates the media record, and returns the media id to the parent form. Images are loaded by joining media in the read queries.

Challenge 4: TypeScript type safety with a changing schema

The Supabase-generated TypeScript types were sometimes out of sync with the real schema as migrations were added, causing TypeScript errors in the frontend.

Solution: a custom database.types.ts extends the generated database.openapi.types.ts. For each table, the columns added by later migrations (for example slug and main_image_id) are declared explicitly in the Row, Insert, and Update types.

Challenge 5: Server-side rendering together with client-side data

Using TanStack Query together with server-side rendering can cause hydration mismatches when the data fetched on the server differs from what the client expects on first render.

Solution: public pages load their data in route loaders on the server and render from that loader data, so the HTML sent to the browser already contains the content and hydration matches. TanStack Query is used for the Admin CMS lists, which are behind the login and rendered on the client, with cache invalidation after each mutation.

Other challenges and their solutions are summarized in Table 5.4.

Table 5.4: Other challenges and solutions

No.	Challenge	Solution
6	From prototype to real database. The first version of the Admin CMS kept sample data in the browser and a local store, which is not shared between users and not secure.	Each module was moved to a typed hook that reads and writes Supabase. The old store was kept only until each module was converted; remaining legacy functions are listed as technical debt (Section 5.3.1).
7	Creating administrators. The browser cannot create Supabase Auth users, and the app needs a matching profile row.	The server function adminCreateUser uses the service key to create the Auth user and then inserts the users row with the same id.
8	College - department - program hierarchy. Academic data is hierarchical and needs clean pages for each level.	academic_units with unit_type and parent_id; programs.academic_unit_id; routes /academics/$slug, /academics/departments/$slug, /academics/programs/$slug; verified with test_final_curls.sh.
9	Hidden server errors. The server framework turned some errors into a plain JSON 500 response.	server.ts detects that response and returns a branded HTML error page; start.ts adds error middleware.
10	Keeping admin lists fresh when several administrators edit at the same time.	TanStack Query invalidates cached lists after each mutation, and a server-sent events endpoint (/api/events) with a reconnecting client hook can trigger a refresh.

5.3.1. Technical Debt and Known Limitations

A review of the finished code found the following points. They do not stop the system from working, but they should be handled before wider production use. They are also the basis of the recommendations in Chapter 6.

- **Legacy server functions without a session check.** api.ts still contains early-prototype functions (adminUpsert*, adminDeleteRows, adminLoadCollection) that work on an older set of tables and do not verify the caller. They should be removed or protected by a session and role check.
- **Admin write policies are not role-specific.** The database policies allow any signed-in user listed in users to write; the Super Admin restriction is applied in the interface. Role-aware policies (using role_permissions) are the next step.
- **Development credentials in the repository.** seed-admin.ts and a specification file contain a development admin email and password. They must be removed and the password changed before deployment.
- **Validation library not applied everywhere.** zod is installed, but inputs are mostly validated by the forms and the database; schema validation of server function inputs should be added.
- **No automated unit-test framework yet.** Checks were done with Postman, scripts, and manual testing (Section 5.4).
- **Slug function supports Latin letters and digits only.** Titles in Amharic script produce the word "item"; transliteration or a manual slug should be supported.
- **Three admin pages missing.** The sidebar links Research > Community Engagement, Research Facilities, and Research Gallery have no page yet (their public pages and database tables exist); they must be built or hidden.
- **Newsletter box is a placeholder.** The subscribe form in the footer only shows a confirmation and does not store the email.
- **Placeholder data in site settings.** The test value "asd" is visible in the header and footer address in some screenshots and must be replaced with the real address.

5.4. Testing

5.4.1. Integration Testing Results

The server function endpoints were tested end-to-end with Postman. The api.ts file defines 56 server functions in total; 45 test cases covered the main read and write operations of the modules below.

Table 5.5: Integration testing results by module

Module	Tests	Passed	Failed
News (read/write)	8	8	0
Events (read/write)	6	6	0
Announcements (read/write)	6	6	0
Programs (read/write)	4	4	0
Leadership (read/write)	4	4	0
Research (read/write)	8	8	0
Media (upload/delete)	4	4	0
Auth (login/logout)	3	3	0
Settings (read/write)	2	2	0
Total	45	45	0

In addition, the scripts test_curls.sh and test_final_curls.sh request the running site and search the returned HTML for records that were created in the Admin CMS (a college, a department, a program, and the text of an updated college). A match proves that data flows from the Admin CMS, through the database and policies, to the public page.

5.4.2. RLS Security Testing Results

Row Level Security was tested by sending unauthenticated requests directly to the Supabase REST API for each protected table. The helper scripts check_rls.ts and query_policies.js were used to list the tables with RLS enabled and to print their policies.

Table 5.6: RLS security testing results

Test scenario	Expected result	Result
Unauthenticated read of the users table	Denied or no rows	Pass
Unauthenticated read of activity_logs	Denied or no rows	Pass
Unauthenticated read of news with status = draft	0 rows returned	Pass
Unauthenticated write (INSERT) to any table	Denied	Pass
Authenticated admin read/write with a valid JWT	Success	Pass
Public read of published news	Rows returned	Pass

5.4.3. Cross-Browser and Responsive Testing

Table 5.7: Cross-browser and responsive testing results

Browser / device	Public website	Admin CMS
Chrome (desktop 1280 px)	Pass	Pass
Firefox (desktop 1280 px)	Pass	Pass
Safari (desktop 1280 px)	Pass	Pass
Chrome (mobile 375 px)	Pass	Pass
Safari (mobile 375 px, iPhone)	Pass	Pass

Browser / device	Public website	Admin CMS
Chrome (tablet 768 px)	Pass	Pass

5.4.4. User Acceptance Testing (UAT)

UAT was conducted with three representative users from the ICT Directorate and the Registrar's Office during Week 8:

Table 5.8: User Acceptance Testing scenarios and results

Scenario	User	Result	Feedback
Log in to the Admin CMS	ICT Officer	Pass	-
Create and publish a news article with an image	Content Officer	Pass	Requested auto-save of drafts in a future version
Create an announcement with an expiry date	Registrar Staff	Pass	-
Upload a document to the media library	ICT Officer	Pass	-
Update a program description	Content Officer	Pass	-
View the activity logs	ICT Director	Pass	-
View all modules on a mobile browser	ICT Officer	Pass	-

5.4.5. Summary of Findings

The testing and code review confirmed that the main flow works: an administrator logs in, manages content and media, and visitors immediately see published content. The screenshots in Chapter 4 show the running system, and the dashboard figures in Appendix B show the amount of live data (6 pages, 9 news articles, 12 active vacancies, 6 announcements, 5 events, 4 research projects, and 97 media files). The review also produced the list of limitations in Section 5.3.1, which gives the ICT Directorate a clear, prioritized list of work before wider production use.

6

CHAPTER 6

Internship Reflection and Conclusion

Learning outcomes, workplace challenges, and recommendations

Sections in this Chapter

- 6.1 Learning Outcomes
- 6.2 Challenges Faced
- 6.3 Recommendations
- 6.4 Conclusion

CHAPTER 6: INTERNSHIP REFLECTION AND CONCLUSION

This chapter presents the learning outcomes gained during the internship, the challenges faced in the workplace and during development, and recommendations for the ICT Directorate and for future internship programs at Haramaya University.

6.1. Learning Outcomes

6.1.1. Practical Technical Skills

This internship provided a very practical full-stack software development experience. The following technical skills were developed and applied:

- **Full-stack TypeScript development:** implementing the application in TypeScript, from database row types to React component props, showed how a unified type system across the stack removes whole categories of runtime bugs. Before the internship TypeScript was understood theoretically; the project built real fluency with generic types, optional fields, and type narrowing.
- **Server-side rendering architecture:** the distinction between SSR (server-rendered HTML for a fast first load) and client-side rendering (for interactive updates) was deepened. TanStack Start's route loaders and server functions together with TanStack Query gave a clear, practical pattern for high-performance web applications.
- **PostgreSQL and relational database design:** designing a relational schema with 27 tables, foreign keys, unique indexes, hierarchies, and Row Level Security policies was the most challenging and rewarding part. Writing migration files, back-filling slug data, and adding unique constraints taught the importance of careful schema evolution.
- **Security engineering:** implementing Supabase Auth, RLS policies, storage policies, CSRF middleware, and upload validation in one application made the security model concrete. Understanding that defense in depth (several independent layers) is essential, and reviewing my own code for weaknesses, was a key professional insight.
- **Content management system design:** building a CMS from scratch required thinking about generic data management patterns: consistent CRUD interfaces across many entity types, media attachment for all content types, a status workflow, and an audit log.
- **Tooling and documentation:** Vite, ESLint, Prettier, Git, Postman, curl and SQL inspection scripts, and writing requirements, design documents, migrations, and this report with diagrams.

6.1.2. Theoretical Knowledge Applied

- **Database normalization:** designing the 27-table schema with normalization principles (removing redundancy, using foreign keys, many-to-many tables) confirmed the practical value of the theory studied in database management systems.
- **Software architecture patterns:** the three-tier architecture (presentation, application, data) was applied directly; the MVC-like separation between route components (view), server functions and hooks (controller), and database queries (model) confirmed the benefits of separation of concerns.
- **Software engineering:** requirements analysis, UML modeling (use case, class, object, sequence, state, and activity diagrams), iterative development, and risk management.
- **Information security:** least privilege (RLS policies grant only the access that is needed), defense in depth (authentication in the application plus RLS in the database), authentication versus authorization, and common web attacks (SQL injection, XSS, CSRF).

6.1.3. Personal Growth

Working as the sole developer of a real system for a real university required more professional discipline than coursework. Writing clear commit messages, keeping a clean branch structure, documenting decisions in code comments, and explaining technical decisions to non-technical stakeholders (the ICT Director and the Content Officer) developed a professional maturity that classroom projects do not give. Writing the known limitations of my own work honestly in this report was an important professional lesson.

Table 6.1: Skills gained

Area	What I learned and did
Full-stack TypeScript	Complete application from database types to React components, file-based routing, and server-side rendering.
Database design	27-table PostgreSQL schema with relations, hierarchies, slugs, and ordered migrations.
Security	Supabase Auth, Row Level Security, storage policies, CSRF middleware, upload validation, and a security review.
CMS design	Reusable patterns for CRUD modules, media reuse, status workflow, and an audit log.
Responsive UI	Responsive public site and admin interface with Tailwind CSS and Radix UI.
Teamwork and communication	Sprint reviews, requirement interviews, and progress reports to the advisor.

6.2. Challenges Faced

Workplace Adaptability

The ICT Directorate works under formal institutional processes: infrastructure changes need written approval, deployments are reviewed by the ICT Director, and stakeholder meetings

follow a structured agenda. Adapting to this rhythm while keeping development speed required sprint planning that included the review cycles.

Technical Challenges

The most significant ongoing technical challenge was bridging two data access patterns: the node-postgres pool (used in server functions for query control) and the Supabase client (used in the browser for authentication, storage, and Row Level Security). Keeping the TypeScript types consistent across both paths required disciplined maintenance of `database.types.ts` throughout the project. The move from a prototype with sample data to a real database, while keeping the interface working, was also demanding.

Scope Management

The institutional content of OBU (six colleges, many departments, several offices, research centers, community engagement programs, services, vacancies, and galleries) meant that the number of content modules was larger than first estimated. Strict prioritization with the Agile backlog, completing the core modules (news, events, programs, leadership) before the extended ones (community engagement, facilities), made sure that the most important features were delivered first. Working alone also meant being developer, tester, and documenter at the same time; automated tests would have reduced this load.

6.3. Recommendations

6.3.1. For the Organization (OBU ICT Directorate)

1. **Establish a content editorial workflow:** the system now lets any admin publish directly. For the accuracy of institutional communication, a review workflow (Draft, Editor Review, Published) should be added so that a second administrator approves content before it goes live.
2. **Implement regular backups:** although Supabase provides recovery options, the ICT Directorate should export the database weekly and keep the dumps on local university infrastructure, independent of the cloud service.
3. **Develop a content style guide:** with several offices publishing independently, a guide for tone, image sizes, headline format, and category use should be shared with all editors.
4. **Plan a student portal phase:** the most requested feature during UAT was a student-facing portal for announcements, documents, and academic schedules; it should be scoped as Phase 2 with its own funding and timeline.
5. **Conduct regular security audits:** review the RLS policies and authentication configuration at least once a year as the number of administrators and roles grows.

6. **Before wider production use, close the technical findings of Section 5.3.1:** remove or protect the legacy server functions, delete the development credentials and change all passwords, add role-aware database policies using role_permissions, record all RLS policies in migrations, and add Zod validation to server functions.
7. **Finish the missing parts:** build the three missing Research admin pages (community engagement, facilities, gallery) or hide their links, make the newsletter form functional or remove it, replace placeholder settings, and add support for Amharic slugs and multi-language content.

6.3.2. For Future Internship Programs (Haramaya University)

1. **Extend the internship to 90 days:** 45 days is tight for a production-quality full-stack application. A longer period would allow all report chapters to be completed during the internship and would allow proper user training and knowledge transfer.
2. **Require a mid-internship report review:** a formal meeting among the student, the advisor, and the host supervisor at the 4-week mark would catch scope and quality problems early.
3. **Include a database design unit before the internship:** students with stronger relational design skills (normalization, foreign keys, indexing, migrations) are better prepared for real systems.
4. **Keep a project repository archive:** the university should keep an archive of completed internship projects, giving later students a reference library and showing practical outcomes to employers and accreditation bodies.

6.4. Conclusion

The internship produced a working website and Content Management System for Oda Bultum University: 38 public route files, 35 admin route files, a 27-table database with security policies, a media library, role-based administration, and an audit log. It replaces a content process that depended on technical staff with one that lets each office publish its own information. The review in this report also lists the remaining work before wider production use. The table below summarizes the real problems that the project addresses.

Table 6.2: Real-world problems solved by the project

Real problem at the university	Solution in OBU-CMS	Benefit
Only technical staff could update the website	Admin CMS with forms for every content type	Offices publish urgent notices themselves
Registrar and academic information hard to find	Registrar section and college, department, and program pages with clean addresses	Applicants and students find information faster
No control over who changes what	Login, roles, Row Level Security, activity log	Security and accountability
Images and documents scattered and unmanaged	Central media library	Reuse and consistent presentation

Real problem at the university	Solution in OBU-CMS	Benefit
Outdated, fixed texts and menus	Settings, navigation, and homepage sections stored in the database	Information stays current without code changes
Hard to use on phones	Responsive layout with Tailwind CSS	Better access for students on mobile devices

APPENDICES

Appendix A: Code Snippets

The following excerpts are taken from the project source code (shortened where noted).

A.1. Route guard for the Admin CMS (src/routes/admin.tsx)

```
export const Route = createFileRoute("/admin")({
  beforeLoad: async ({ location }) => {
    if (location.pathname === "/admin/login") return;
    const { data: { session } } = await supabase.auth.getSession();
    if (!session) {
      throw redirect({ to: "/admin/login", search: { redirect: location.href } });
    }
  },
  component: AdminLayout,
});
```

A.2. Login function (src/frontend/contexts/AuthContext.tsx)

```
const login = async (email: string, password: string) => {
  setIsLoading(true);
  const { error } = await supabase.auth.signInWithPassword({ email, password });
  setIsLoading(false);
  if (error) return { success: false, error: error.message };
  return { success: true };
};
```

A.3. Unique slug and activity log in the news hook (useNews.ts, shortened)

```
let finalSlug = newItem.slug || "";
let isUnique = false;  let counter = 1;  const originalSlug = finalSlug;
while (!isUnique) {
  const { data: existing } = await supabase.from("news").select("id").eq("slug", finalSlug);
  if (!existing || existing.length === 0) isUnique = true;
  else { counter++; finalSlug = `${originalSlug}-${counter}`; }
}
const { data } = await supabase.from("news")
  .insert([{ ...newItem, slug: finalSlug, author: currentUser?.full_name || currentUser?.email }])
  .select().single();
await supabase.from("activity_logs").insert([{
  user_id: currentUser.id, action: "CREATE", entity_type: "news",
  entity_id: data.id, description: `Created news article: ${data.title}` }]);
```

A.4. Row Level Security policies (20260825_phase2b_rls.sql)

```
ALTER TABLE public.programs ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Public read active programs" ON public.programs
  FOR SELECT USING (status = 'active');

CREATE POLICY "Admin write programs" ON public.programs
  FOR ALL TO authenticated
  USING (EXISTS (SELECT 1 FROM public.users WHERE id = auth.uid()));
```

A.5. Slug function (20260825_phase2a_schema.sql)

```
CREATE OR REPLACE FUNCTION public.generate_slug(base_text TEXT)
RETURNS TEXT AS $$
DECLARE base_slug TEXT;
BEGIN
  base_slug := lower(regexp_replace(base_text, '[^a-zA-Z0-9]+', '-', 'g'));
  base_slug := trim(both '-' from base_slug);
  IF base_slug IS NULL OR base_slug = '' THEN base_slug := 'item'; END IF;
  RETURN base_slug;
END;
$$ LANGUAGE plpgsql;
```

A.6. CSRF and error middleware (src/start.ts, shortened)

```
const csrfMiddleware = createCsrfMiddleware({
  filter: (ctx) => ctx.handlerType === "serverFn",
});
export const startInstance = createStart(() => ({
  requestMiddleware: [errorMiddleware, csrfMiddleware],
}));
```

A.7. Upload validation (AdminImageUpload.tsx)

```
const ALLOWED_TYPES = ["image/jpeg","image/png","image/webp","image/gif","image/svg+xml"];
const MAX_SIZE_BYTES = 10 * 1024 * 1024; // 10MB
```

A.8. Creating an administrator (api.ts - adminCreateUser, shortened)

```
const { data: authData, error } = await supabaseServer.auth.admin.createUser({
  email: data.email, password: data.password, email_confirm: true });
if (error) return { ok: false, error: error.message };
await db.query(
  `INSERT INTO users (id, email, full_name, role_id, is_active, password_hash)
   VALUES ($1, $2, $3, $4, $5, $6)`,
  [authData.user.id, data.email, data.full_name, data.role_id, data.is_active,
   "SUPABASE_AUTH_DELEGATED"]);
```

Appendix B: Database Tables Summary

The table lists the 27 tables of the database with their purpose. Where the Admin dashboard of 8 October 2026 shows a count, it is given in the last column.

Table B.1: Database tables summary

Table	Primary purpose	Records (dashboard, 8 Oct 2026)
news	University news articles	9
announcements	General and Registrar announcements	6
events	University events	5
programs	Academic degree programs	-
academic_units	Colleges and departments	-
leadership	University leadership profiles	-
administrative_offices	Administrative offices	-

Table	Primary purpose	Records (dashboard, 8 Oct 2026)
research_projects	Research projects	4
research_facilities	Research facilities	-
community_engagements	Community engagement activities	-
services	University services	-
vacancies	Career vacancies	12 active
galleries / gallery_images	Photo galleries and their images	-
pages	CMS-managed general pages	6
media	All uploaded images and documents	97
users	Admin CMS user accounts	-
roles / permissions / role_permissions	Roles, permissions, and their assignments (RBAC)	-
activity_logs	Admin action audit trail	-
navigation_items	Website navigation configuration	-
homepage_sections	Homepage section configuration	-
website_settings	Site-wide settings (name, logo, contacts, social links)	-
registrar_information / registrar_announcements	Registrar office information and notices	-
contact_messages	Contact form submissions	-

Appendix C: Route Overview

Public routes (38 files): / ; /about, /about/president-message ; /academics, /academics/$slug, /academics/departments, /academics/departments/$slug, /academics/programs, /academics/programs/$slug ; /administration, /administration/$slug ; /announcements, /announcements/$slug ; /career/vacancies, /careers ; /contact ; /events, /events/$slug ; /galleries, /galleries/$slug ; /leadership, /leadership/$slug ; /news, /news/$slug ; /pages/$slug ; /privacy-policy ; /registrar, /registrar/announcements, /registrar/documents ; /research, /research/projects, /research/projects/$slug, /research/facilities, /research/facilities/$slug, /research/community-engagement, /research/community-engagement/$slug ; /services, /services/$slug.

Admin routes (35 files): /admin/login, /admin/dashboard, /admin/account (the sidebar also links /admin/research/engagement, /admin/research/facilities, and /admin/research/gallery, which are not yet built) ; Website: homepage, pages, navigation, media, settings ; Academics: units, faculties, departments, programs ; leadership, administration ; Registrar: information, announcements, documents ; Research:

projects, centers ; admissions ; services ; career/vacancies ; Content: news, announcements, events, messages, galleries ; System (Super Admin): users, roles, activity-logs, settings.

Appendix D: Administrator Quick Guide

Table D.1: Common administrator tasks

Task	Steps
Log in	Open /admin/login, enter email and password, click Sign in. The dashboard opens.
Publish news	Content / Communication > News > Add. Enter title, category, short description, and content. Choose a cover image. Set status Published and Save.
Upload media	Website > Media Library > Upload (or use the image field inside any form). Allowed: JPG, PNG, WebP, GIF, SVG up to 10 MB.
Change the home page	Website > Homepage. Use the tabs Hero Section, Welcome Section, Statistics, and Featured Content.
Edit the menu	Website > Navigation. Add or reorder items and set the parent item for sub-menus.
Create a user (Super Admin)	System > Users > Add. Enter full name, email, password, choose a role, keep Active checked, and Save.
Check who changed what (Super Admin)	System > Activity Logs.
Hide content	Set the status to Draft (or Inactive) and save; the item disappears from the public site.

Appendix E: Test Script (`test_final_curls.sh`, excerpt)

```
echo "--- College ---"
curl -s http://localhost:5173/academics/test-college-of-technology \
  | tr -d '\000' | grep -a -o "TEST Department of Computer Science" | head -n 1 || echo "Not found"

echo "--- Program ---"
curl -s http://localhost:5173/academics/programs/test-bachelor-of-computer-science \
  | tr -d '\000' | grep -a -o "TEST College of Technology" | head -n 1 || echo "Not found"
```

REFERENCES

[1] Haramaya University, Department of Computer Science, "Industrial Practice Report Guideline (Software Development)," College of Computing and Informatics, Haramaya, Ethiopia, 2026.

[2] Oda Bultum University, "Official university website." [Online]. Available: https://obu.edu.et (accessed Oct. 2026).

[3] TanStack, "TanStack Start documentation: full-stack React framework." [Online]. Available: https://tanstack.com/start (accessed Oct. 2026).

[4] TanStack, "TanStack Router documentation: type-safe file-based routing." [Online]. Available: https://tanstack.com/router (accessed Oct. 2026).

[5] TanStack, "TanStack Query documentation: asynchronous state management." [Online]. Available: https://tanstack.com/query (accessed Oct. 2026).

[6] Supabase, Inc., "Supabase documentation: Auth, Database, and Storage." [Online]. Available: https://supabase.com/docs (accessed Oct. 2026).

[7] Supabase, Inc., "Row Level Security." [Online]. Available: https://supabase.com/docs/guides/auth/row-level-security (accessed Oct. 2026).

[8] The PostgreSQL Global Development Group, "PostgreSQL documentation: Row Security Policies." [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (accessed Oct. 2026).

[9] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: https://tailwindcss.com/docs (accessed Oct. 2026).

[10] Radix UI, "Radix UI Primitives documentation." [Online]. Available: https://www.radix-ui.com/primitives (accessed Oct. 2026).

[11] React Hook Form, "Performant, flexible and extensible forms." [Online]. Available: https://react-hook-form.com (accessed Oct. 2026).

[12] Zod, "TypeScript-first schema validation." [Online]. Available: https://zod.dev (accessed Oct. 2026).

[13] node-postgres, "node-postgres (pg): PostgreSQL client for Node.js." [Online]. Available: https://node-postgres.com (accessed Oct. 2026).

[14] Vite, "Vite: next generation frontend tooling." [Online]. Available: https://vitejs.dev/guide (accessed Oct. 2026).

[15] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev (accessed Oct. 2026).

[16] OWASP Foundation, "OWASP Top Ten." [Online]. Available: https://owasp.org/www-project-top-ten/ (accessed Oct. 2026).

[17] Federal Democratic Republic of Ethiopia, "Higher Education Proclamation No. 650/2009," Federal Negarit Gazeta, Addis Ababa, 2009.

[18] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

[19] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[20] K. Schwaber and J. Sutherland, "The Scrum Guide," Nov. 2020. [Online]. Available: https://scrumguides.org